Pengembangan Aplikasi Berbasis Platform

Disusun Kelompok Black Sweet :

Felix Andreas Sarumpaet – 231121463

Wicho Fairuz Al Alim - 231121300

Samuel Yogi Nainggolan - 231121217

Agung Anugrah Sitohang - 231121561

Tengku Ahmad Sajali - 231121794

FAKULTAS INFORMATIKA

UNIVERSITAS MIKROSKIL

MEDAN

2026

DAFTAR ISI

1. Latar Belakang 4
1.1 Gambaran Umum 4
1.2 Rumusan Masalah 4
1.3 Tujuan 4
1.4 Manfaat 5
1.5 Ruang Lingkup 5
2. Requirement / Problem 6
2.1 Problem Statement 6
2.2 Pemangku Kepentingan dan Pengguna 6
3. API Integration 7
3.1 Pola Integrasi 7
3.2 Lapisan API di Frontend 7
3.3 Kode Inti Integrasi 8
3.4 Hook useFetch 8
3.5 Pemetaan Halaman ke Endpoint 8
4. Aplikasi Klien Web 9
4.1 Web Client 9
5. API Documentation 11
5.1 Informasi Umum 11
5.2 Ringkasan Endpoint 11
5.3 Auth 12
5.4 Stats 14
5.5 Customers 14
5.6 Services 14
5.7 Orders 15
6. Use Case Diagram 17
6.1 Aktor 17
6.2 Diagram 17
6.3 Skenario Use Case 18
7. UI Design 20
7.1 Prinsip Desain 20
7.2 Struktur dan Alur Pengguna 21
7.3 Identitas Visual 21
7.4 Komponen Antarmuka 22
7.5 Desain Halaman 23
8. Flowchart 28
8.1 Alur Status Pesanan 28
8.2 Flowchart Login 29
8.3 Flowchart Registrasi Akun 30
8.4 Flowchart Lupa Password 31
8.5 Flowchart Pembuatan Pesanan 32
8.6 Flowchart Ubah Status Pesanan 33
8.7 Flowchart Hapus Pelanggan / Layanan 34
8.8 Flowchart Pemrosesan Request di Server 35

9. Data Model 36
9.1 Entity Relationship Diagram 36
9.2 Kamus Data 36
9.3 Domain Nilai (Enumerasi) 38
9.4 Indeks, Integritas, dan Data Awal 39
9.5 Analisis Normalisasi 39
9.6 Agregasi Data untuk Dashboard 39
9.7 Model Data di Sisi Klien (TypeScript) 40
10. Architecture Diagram 41
10.1 Arsitektur Sistem 41
10.2 Lapisan Backend 41
10.3 Alur Pembuatan Pesanan 42
10.4 Arsitektur Keamanan 42
10.5 Arsitektur Penerapan 43
11. Integration Doc 44
11.1 Prasyarat 44
11.2 Langkah Menjalankan 44
11.3 Konfigurasi 44
11.4 Alur Integrasi Autentikasi 45
11.5 Contoh Pemanggilan dengan cURL 45
11.6 Contoh Integrasi Klien JavaScript 45
11.7 Pengujian Integrasi 46
11.8 Penanganan Masalah Umum 46
12. Feature Depth 47
12.1 Fitur Lanjutan Sisi Server 47
12.2 Fitur Lanjutan Sisi Klien 47
12.3 Aturan Bisnis Utama 48
13. Penutup 49
13.1 Kesimpulan 49

1. Latar Belakang

1.1 Gambaran Umum

Bagi banyak usaha laundry kiloan, buku nota masih menjadi andalan untuk mencatat cucian pelanggan. Setiap pesanan harus ditulis lengkap: berat cucian, jenis layanan, harga, tanggal masuk, dan perkiraan kapan cucian bisa diambil. Selama pelanggannya masih sedikit, cara ini masih bisa diatasi. Begitu pelanggan bertambah, kesalahan mulai bermunculan: hitungan harga meleset, nota hilang, cucian tertukar, dan pemilik kesulitan menjawab pertanyaan sesederhana "cucian saya sudah sampai tahap mana?".

LaundryBlackSweet kami bangun untuk menjawab persoalan itu. Aplikasi ini menyatukan pencatatan pesanan, data pelanggan, daftar layanan beserta harga per kilogram, dan ringkasan pendapatan (hari ini, bulan ini, dan bulan lalu) dalam satu sistem. Di belakangnya ada satu REST API (Node.js + Express 5 dengan basis data SQLite) yang menjadi satu-satunya sumber data. Antarmuka penggunanya berupa aplikasi web (Web Client) yang dibangun dengan React dan memakai API tersebut.

1.2 Rumusan Masalah

1. Bagaimana mencatat pesanan laundry dengan cepat dan akurat tanpa menghitung harga secara manual?
2. Bagaimana memantau tahapan pengerjaan cucian, dari diterima sampai diambil, secara real time?
3. Bagaimana menyimpan data pelanggan dan layanan agar tidak ganda dan tidak rusak ketika ada yang dihapus?
4. Bagaimana menampilkan ringkasan pendapatan harian dan bulanan secara otomatis?
5. Bagaimana menjaga data usaha agar hanya pengguna terdaftar yang bisa mengaksesnya?
6. Bagaimana memisahkan logika bisnis di server dari tampilan di klien web, sehingga aturan bisnis hanya ada di satu tempat?

1.3 Tujuan

1. Membangun sistem client-server yang terdiri dari REST API Express, basis data SQLite, dan klien web (React).
2. Menghitung total harga di server sehingga hasilnya konsisten dan tidak bisa diubah-ubah dari sisi klien.
3. Menyediakan alur status pesanan yang jelas, lengkap dengan struk yang bisa dicetak.

4. Menyediakan autentikasi yang sederhana tetapi tetap aman: password di-hash, token bertanda tangan, dan percobaan login dibatasi.
5. Mendokumentasikan sistem secara lengkap (analisis, desain, API, integrasi) supaya mudah dipelihara dan dikembangkan orang lain.

1.4 Manfaat

1. Pemilik usaha: pencatatan lebih rapi, pendapatan mudah dipantau, dan risiko salah hitung berkurang.
2. Karyawan atau kasir: input pesanan lebih cepat, dan struk bisa dicetak lewat peramban (browser) di perangkat apa pun yang tersedia.
3. Pelanggan: status cucian lebih jelas dan ada struk tercetak sebagai bukti.
4. Pengembang dan mahasiswa: contoh utuh penerapan REST API, validasi, autentikasi, dan integrasi klien web dengan server.

1.5 Ruang Lingkup

Sistem ini mencakup registrasi dan login pengguna, pemulihan password lewat pertanyaan keamanan, dashboard ringkasan, pengelolaan pesanan (membuat, mencari, menyaring, mengubah status, menghapus, dan mencetak struk), serta pengelolaan data pelanggan dan layanan (tambah, lihat, ubah, hapus). Ada beberapa hal yang sengaja tidak dimasukkan: pembayaran online, notifikasi WhatsApp/SMS, ekspor laporan, dan peran pengguna bertingkat. Semua pengguna terdaftar memiliki hak akses yang sama.

Perlu dicatat bahwa dokumen ini hanya membahas Web Client (React), yang sudah diimplementasikan penuh dan terhubung ke REST API.

2. Requirement / Problem

2.1 Problem Statement

Dari kondisi di lapangan, ada delapan masalah yang ingin kami selesaikan. Berikut masing-masing masalah dan cara LaundryBlackSweet menjawabnya:

1. **Harga dihitung manual dan sering keliru.** Server yang menghitung total_price = round(harga per kg × berat) saat pesanan dibuat, jadi pengguna tidak perlu menghitung sendiri.
2. **Status cucian tidak terpantau.** Ada enam tahap status yang bisa diganti lewat dropdown pada tabel pesanan, dan dashboard menghitung berapa pesanan yang masih aktif dan yang siap diambil.
3. **Data pelanggan ganda atau tidak tercatat.** Pelanggan dicari berdasarkan nama dan nomor HP; kalau belum ada, sistem membuatnya otomatis (findOrCreate).
4. **Nota kertas hilang.** Struk digital bisa dicetak kapan saja dari modal Struk (window.print).
5. **Pendapatan sulit direkap.** Endpoint /api/stats merangkum pendapatan hari ini, bulan ini, dan bulan lalu.
6. **Data bisa dibuka siapa saja.** Semua endpoint /api, kecuali auth dan health, mewajibkan token Bearer.
7. **Pelanggan atau layanan terhapus padahal punya riwayat.** Penghapusan ditolak dengan status 409 selama datanya masih dipakai pada pesanan.
8. **Pencatatan terkunci di satu perangkat.** Satu API terpusat menyimpan seluruh data di server sehingga bisa dibuka dari perangkat mana pun lewat peramban dan selalu sinkron.

2.2 Pemangku Kepentingan dan Pengguna

Sistem ini dipakai oleh tiga kelompok pengguna dengan kebutuhan yang berbeda. Pemilik laundry ingin melihat ringkasan pendapatan dan jumlah cucian yang sedang diproses dalam sekali pandang. Karyawan atau kasir menginput pesanan berkali-kali sepanjang hari, sehingga mereka butuh formulir yang singkat, total harga yang langsung terlihat, dan struk yang mudah dicetak. Terakhir, ada pengguna yang membuka sistem lewat peramban di perangkat berlayar kecil, jadi tampilannya harus menyesuaikan ukuran layar.

3. API Integration

Bab ini menjelaskan bagaimana Web Client terhubung ke backend. LaundryBlackSweet memakai REST API internal buatan sendiri (Express) sebagai satu-satunya sumber data. Sistem tidak memanggil layanan pihak ketiga seperti payment gateway atau SMS gateway.

3.1 Pola Integrasi

Seluruh komunikasi berjalan lewat HTTP/1.1 dengan format JSON (Content-Type: application/json). Untuk memanggil API, Web Client memakai axios (frontend/src/api/axiosClient.ts) dengan batas waktu 10 detik. Alamat dasarnya diambil dari import.meta.env.VITE_API_URL pada klien web, dengan nilai bawaan http://localhost:3000/api.

Autentikasi memakai header Authorization: Bearer <token> yang disisipkan otomatis oleh request interceptor. Token disimpan di localStorage browser dengan kunci laundryblacksweet_token. Bila server membalas 401 di luar rute /auth/, response interceptor menghapus token dan mengirim event "auth:logout" sehingga aplikasi kembali ke halaman login.

Respons sukses selalu berbentuk { "message": string, "data": ... }, sedangkan respons error berbentuk { "message": string } dengan status HTTP yang sesuai (400, 401, 404, 409, 429, atau 500). Terakhir, backend memakai cors() supaya frontend di port 5173 bisa memanggil backend di port 3000.

3.2 Lapisan API di Frontend

Setiap domain punya modul sendiri di frontend/src/api. Komponen halaman tidak memanggil axios secara langsung, melainkan lewat fungsi-fungsi di modul berikut:

1. auth.api.ts berisi login, register, getSecurityQuestion, resetPassword, dan getMe, yang memanggil /auth/login, /auth/register, /auth/forgot/question, /auth/forgot/reset, dan /auth/me.
2. order.api.ts berisi getOrders, createOrder, updateOrderStatus, dan deleteOrder, yang memanggil GET/POST /orders, PATCH /orders/:id/status, dan DELETE /orders/:id.
3. customer.api.ts menangani operasi CRUD pelanggan lewat /customers dan /customers/:id.
4. service.api.ts menangani operasi CRUD layanan lewat /services dan /services/:id.

5. stats.api.ts berisi getStats, yang memanggil GET /stats.

3.3 Kode Inti Integrasi

Berikut konfigurasi klien axios beserta interceptor token (ringkasan dari axiosClient.ts):

```
const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_URL ?? "http://localhost:3000/api",
  headers: { "Content-Type": "application/json" },
  timeout: 10000,
});

axiosClient.interceptors.request.use((config) => {
  const token = localStorage.getItem(TOKEN_KEY);
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Dan ini contoh pemanggilan dari lapisan API (order.api.ts):

```
export const updateOrderStatus = async (id: number, status: OrderStatus) => {
  const res = await axios.patch<ApiResponse<Order>>(`/orders/${id}/status`, { status });
  return res.data.data;
};
```

3.4 Hook useFetch

Hook useFetch(fetcher) membungkus pemanggilan API menjadi state { data, loading, error } dan menyediakan dua fungsi: reload, untuk memuat ulang tanpa layar loading, dan retry, untuk memuat ulang dengan layar loading. Hook ini dipakai di halaman Beranda, Pesanan, Pelanggan, dan Layanan, sehingga pola loading, error, dan data terasa seragam di semua halaman.

3.5 Pemetaan Halaman ke Endpoint

Pemetaan halaman ke endpoint sebenarnya cukup lugas. AuthPage memanggil POST /auth/login, /auth/register, /auth/forgot/question, dan /auth/forgot/reset, sedangkan App memeriksa sesi lewat GET /auth/me. DashboardPage membutuhkan GET /stats dan GET /orders (untuk delapan pesanan terbaru).

OrdersPage adalah halaman yang paling banyak memakai API: GET /orders, GET /customers, GET /services, POST /orders, PATCH /orders/:id/status, dan DELETE /orders/:id. CustomersPage memakai GET/POST /customers serta PUT/DELETE /customers/:id, dan ServicesPage memakai pola yang sama pada /services.

4. Aplikasi Klien Web

4.1 Web Client

4.1.1 Teknologi

Web Client dibangun dengan React ^19.3.1 dan react-dom ^19.3.1 memakai bahasa TypeScript ~6.0.3. Build tool-nya adalah Vite ^8.3.4 dengan @vitejs/plugin-react dan React Compiler (babel-plugin-react-compiler). Untuk HTTP client dipakai axios ^1.21.0, dan untuk linter dipakai oxlint ^1.88.0. Styling ditulis dengan CSS murni (src/index.css), termasuk aturan @media print untuk struk. Versi-versi tersebut mengikuti package.json.

4.1.2 Struktur Folder

```
frontend/src/
  api/         axiosClient.ts, auth/order/customer/service/stats .api.ts
  components/  Navbar, Modal, ConfirmDialog, Toast, StateViews, StatusBadge,
               OrderForm, CustomerForm, ServiceForm, ReceiptModal
  hooks/       useFetch.ts
  pages/       AuthPage, DashboardPage, OrdersPage, CustomersPage, ServicesPage
  types/       api, auth, customer, order, service, stats (.ts)
  utils/       format.ts, orderStatus.ts, shop.ts, error.ts
  App.tsx, main.tsx, index.css
```

4.1.3 Navigasi dan State

1. Aplikasi tidak memakai router eksternal. Halaman aktif (dashboard, orders, customers, services) disimpan di hash URL (#orders dan seterusnya) sehingga tidak hilang saat halaman di-refresh.
2. Jika pengguna belum login, App menampilkan AuthPage. Kalau sudah ada token tersimpan, App lebih dulu memeriksa sesi lewat GET /auth/me sambil menampilkan layar "Memeriksa sesi...".
3. Notifikasi sukses atau gagal muncul sebagai toast yang hilang sendiri setelah 3,5 detik.
4. Tampilan memakai format lokal: rupiah (Intl.NumberFormat id-ID), berat dalam kg, dan tanggal berbahasa Indonesia.

4.1.4 Daftar Halaman

Web Client terdiri dari lima halaman. Halaman Masuk / Daftar / Lupa password (AuthPage) menyatukan tiga mode dalam satu halaman: login dengan username dan password; pembuatan akun (nama lengkap, username, password, ulangi password, pertanyaan keamanan, dan jawaban); serta pemulihan password dua langkah.

Beranda (DashboardPage) menampilkan tiga kartu pendapatan (hari ini, bulan ini yang disorot berlatar biru, bulan lalu), tiga kartu hitungan (sedang diproses, siap diambil, pelanggan), dan tabel delapan pesanan terbaru. Halaman Pesanan (OrdersPage) adalah halaman kerja utama: ada tombol Pesanan baru, kolom pencarian, filter status, tabel pesanan dengan dropdown ubah status, serta tombol Struk dan Hapus (yang meminta konfirmasi lewat dialog).

Halaman Pelanggan (CustomersPage) menyediakan pencarian berdasarkan nama, nomor HP, atau alamat, tabel pelanggan lengkap dengan jumlah pesanannya, dan tambah/ubah/hapus lewat modal. Halaman Layanan (ServicesPage) serupa: tabel layanan (nama, keterangan, harga per kg, jumlah pemakaian) dengan tambah/ubah/hapus lewat modal.

4.1.5 Komponen Pendukung

Beberapa komponen dipakai berulang di banyak halaman. Navbar memuat menu empat halaman, identitas pengguna, dan tombol keluar. Modal dan ConfirmDialog menjadi wadah formulir dan dialog konfirmasi penghapusan. OrderForm adalah formulir pesanan dengan isian nama (dengan saran pelanggan yang sudah ada), nomor HP, layanan, berat, tanggal masuk, estimasi diambil, metode pembayaran, dan catatan, sedangkan CustomerForm dan ServiceForm adalah formulir tambah/ubah untuk pelanggan dan layanan.

ReceiptModal menampilkan struk berisi identitas usaha, detail pelanggan, layanan, berat, dan total, dengan tombol Cetak struk yang memanggil window.print(). Sisanya adalah komponen kecil: StatusBadge untuk lencana status, StateViews untuk tampilan loading, kosong, dan error, serta Toast untuk notifikasi.

5. API Documentation

5.1 Informasi Umum

API berjalan di http://localhost:3000 (port bisa diubah lewat env PORT) dengan prefix /api. Semua request dan respons memakai JSON, dan request body harus bertipe Content-Type: application/json. Kecuali endpoint yang ditandai Publik, setiap endpoint mewajibkan header Authorization: Bearer <token>, dan token berlaku selama 12 jam.

Respons sukses berbentuk { "message": "...", "data": ... }, sedangkan respons error berbentuk { "message": "..." }. Kode status HTTP yang dipakai adalah sebagai berikut:

1. **200:** berhasil (GET, PUT, PATCH, DELETE, login, reset password).
2. **201:** data berhasil dibuat (POST pelanggan, layanan, pesanan, dan registrasi).
3. **400:** validasi gagal, ID tidak valid, atau JSON rusak.
4. **401:** token tidak ada, tidak valid, atau kedaluwarsa; bisa juga karena kredensial atau jawaban keamanan salah.
5. **404:** data atau endpoint tidak ditemukan.
6. **409:** konflik, misalnya username sudah dipakai atau pelanggan/layanan masih dipakai pesanan.
7. **429:** terlalu banyak percobaan (login/register/forgot lebih dari 10 kali per 15 menit per IP).
8. **500:** kesalahan tak terduga di server.

5.2 Ringkasan Endpoint

Tabel 5.1 Ringkasan endpoint

Method	Endpoint	Auth	Keterangan
GET	/api/health	Publik	Cek status server
POST	/api/auth/login	Publik	Masuk, mengembalikan token
POST	/api/auth/register	Publik	Buat akun baru (langsung login)
POST	/api/auth/forgot/question	Publik	Ambil pertanyaan keamanan
POST	/api/auth/forgot/reset	Publik	Atur ulang password
GET	/api/auth/me	Ya	Profil pengguna dari token
GET	/api/stats	Ya	Ringkasan dashboard
GET	/api/customers	Ya	Daftar pelanggan (+ order_count)
POST	/api/customers	Ya	Tambah pelanggan
GET	/api/customers/:id	Ya	Detail pelanggan

Method	Endpoint	Auth	Keterangan
PUT	/api/customers/:id	Ya	Ubah pelanggan
DELETE	/api/customers/:id	Ya	Hapus pelanggan
GET	/api/services	Ya	Daftar layanan (+ order_count)
POST	/api/services	Ya	Tambah layanan
GET	/api/services/:id	Ya	Detail layanan
PUT	/api/services/:id	Ya	Ubah layanan
DELETE	/api/services/:id	Ya	Hapus layanan
GET	/api/orders	Ya	Daftar pesanan (join pelanggan dan layanan)
POST	/api/orders	Ya	Buat pesanan
GET	/api/orders/:id	Ya	Detail pesanan
PATCH	/api/orders/:id/status	Ya	Ubah status pesanan
DELETE	/api/orders/:id	Ya	Hapus pesanan

5.3 Auth

POST /api/auth/login

Body: username (string, wajib), password (string, wajib).

```
// Request
{ "username": "admin", "password": "admin123" }
// Response 200
{
  "message": "Login berhasil",
  "data": {
    "token": "<base64url-payload>.<signature>",
    "user": { "id": 1, "username": "admin", "full_name": "Administrator" }
  }
}
```

Error: 400 "Username dan password wajib diisi"; 401 "Username atau password salah"; 429 "Terlalu banyak percobaan. Coba lagi beberapa menit lagi."

POST /api/auth/register

Body: full_name (wajib); username (wajib, 3-30 karakter: huruf, angka, titik, atau garis bawah; harus unik); password (minimal 6 karakter); security_question (wajib); security_answer (wajib, disimpan sebagai hash dengan huruf dinormalisasi ke huruf kecil).

```
// Response 201
{ "message": "Akun berhasil dibuat",
  "data": { "token": "...", "user": { "id": 2, "username": "budi", "full_name": "Budi" } } }
```

Error: 400 (validasi); 409 "Username sudah dipakai".

POST /api/auth/forgot/question

```
// Request
{ "username": "admin" }
// Response 200
{ "message": "OK", "data": { "question": "Nama usaha laundry Anda?" } }
```

Error: 404 "Username tidak ditemukan"; 400 "Akun ini belum memiliki pertanyaan keamanan".

POST /api/auth/forgot/reset

```
// Request
{ "username": "admin", "security_answer": "laundryblacksweet", "new_password": "passwordBaru1" }
// Response 200
{ "message": "Password berhasil diganti. Silakan masuk." }
```

Error: 400 "Password minimal 6 karakter"; 401 "Jawaban keamanan salah".

GET /api/auth/me

```
// Response 200
{ "message": "OK", "data": { "id": 1, "username": "admin", "full_name": "Administrator" } }
```

Error: 401 "Sesi berakhir, silakan login kembali".

5.4 Stats

GET /api/stats

```
// Response 200
{
  "message": "Ringkasan berhasil diambil",
  "data": {
    "active_orders": 3,        // status bukan completed / picked_up
    "ready_orders": 1,         // status completed
    "today_revenue": 18000,
    "month_revenue": 159000,
    "last_month_revenue": 0,
    "total_customers": 5
  }
}
```

5.5 Customers

Body: name (wajib); phone (wajib, harus cocok dengan regex ^[0-9+\\-\\s]{8,16}$); address (opsional).

POST /api/customers

```
// Request
{ "name": "Rizky Pratama", "phone": "081377788899", "address": "Jl. Pattimura No. 9, Medan" }
// Response 201
{ "message": "Pelanggan berhasil ditambahkan",
  "data": { "id": 6, "name": "Rizky Pratama", "phone": "081377788899",
            "address": "Jl. Pattimura No. 9, Medan",
            "created_at": "2026-11-09 03:00:00", "order_count": 0 } }
```

1. GET /api/customers: 200 dengan daftar pelanggan (urut id terbaru).
2. GET /api/customers/:id: 200, atau 404 "Pelanggan tidak ditemukan".
3. PUT /api/customers/:id: 200 "Pelanggan berhasil diperbarui" (body sama dengan POST).
4. DELETE /api/customers/:id: 200 "Pelanggan berhasil dihapus"; 409 "Pelanggan tidak bisa dihapus karena masih memiliki riwayat pesanan".

5.6 Services

Body: service_name (wajib); price_per_kg (wajib, angka lebih dari 0); description (opsional).

POST /api/services

```
// Request
{ "service_name": "Cuci Setrika", "price_per_kg": 8000, "description": "Cuci, kering, setrika." }
// Response 201
{ "message": "Layanan berhasil ditambahkan",
  "data": { "id": 6, "service_name": "Cuci Setrika", "price_per_kg": 8000,
            "description": "Cuci, kering, setrika.", "created_at": "...", "order_count": 0 } }
```

1. GET /api/services (urut id naik), GET /api/services/:id, PUT /api/services/:id, DELETE /api/services/:id.
2. DELETE: 409 "Layanan tidak bisa dihapus karena sudah dipakai pada pesanan".

5.7 Orders

POST /api/orders

Tabel 5.2 Aturan field pesanan

Field	Wajib	Aturan
customer_name	Ya	String tidak kosong
customer_phone	Ya	Regex ^[0-9+\-\s]{8,16}$
service_id	Ya	Bilangan bulat lebih dari 0 dan layanan harus ada (404 bila tidak)
weight	Ya	Angka lebih dari 0 dan maksimal 1000 (kg)
status	Tidak	Salah satu enum status; bawaan received
payment_method	Tidak	cash | transfer | qris; bawaan cash
order_date	Tidak	YYYY-MM-DD; bila kosong memakai waktu sekarang
pickup_date	Tidak	YYYY-MM-DD
notes	Tidak	String

```
// Request
{
 "customer_name": "Adi Wicho Fairuz", "customer_phone": "081234567801",
 "service_id": 2, "weight": 4.5, "order_date": "2026-11-09",
 "pickup_date": "2026-11-11", "payment_method": "qris", "notes": "Pisahkan pakaian putih"
}
// Response 201
{
 "message": "Pesanan berhasil dibuat",
 "data": {
  "id": 6, "customer_id": 1, "service_id": 2, "weight": 4.5,
  "total_price": 36000, "status": "received",
  "order_date": "2026-11-09", "pickup_date": "2026-11-11",
  "payment_method": "qris", "notes": "Pisahkan pakaian putih",
  "customer_name": "Adi Wicho Fairuz", "customer_phone": "081234567801",
  "service_name": "Cuci Setrika", "price_per_kg": 8000
 }
}
```

Error 400: "Nama pelanggan wajib diisi", "Nomor HP tidak valid (8-16 digit angka)", "Layanan wajib dipilih", "Berat cucian harus antara 0 dan 1000 kg", "Status pesanan tidak valid", "Metode pembayaran tidak valid", "Format tanggal tidak valid". Error 404: "Layanan tidak ditemukan".

GET /api/orders dan GET /api/orders/:id

Mengembalikan objek pesanan seperti di atas (daftar diurutkan id terbaru). Detail yang tidak ditemukan menghasilkan 404 "Pesanan tidak ditemukan".

PATCH /api/orders/:id/status

```
// Request
{ "status": "washing" }
// Response 200
{ "message": "Status pesanan berhasil diperbarui", "data": { ...pesanan... } }
```

Nilai status: received, washing, drying, ironing, completed, picked_up. Error: 400 "Status pesanan tidak valid"; 404 "Pesanan tidak ditemukan".

DELETE /api/orders/:id

```
// Response 200
{ "message": "Pesanan berhasil dihapus" }
```

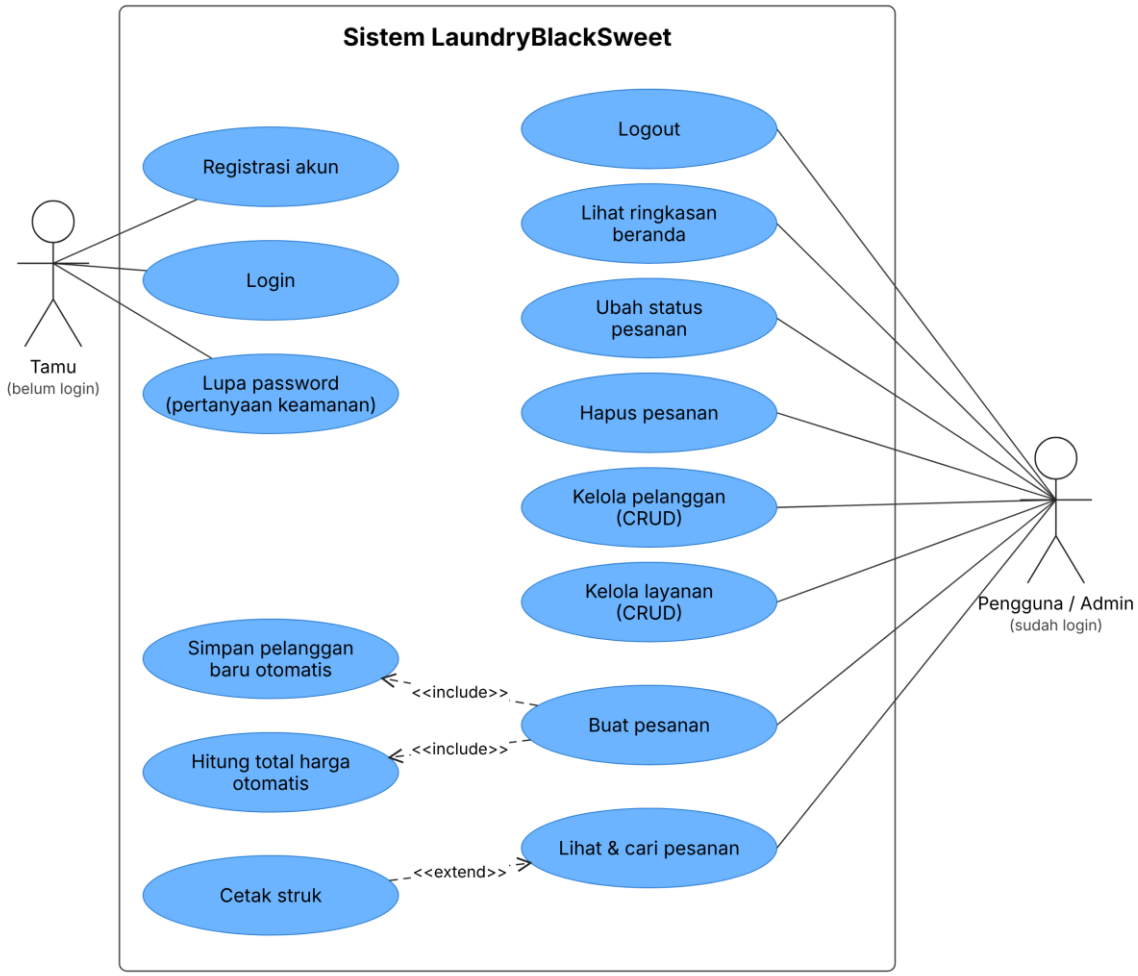
Error: 404 "Pesanan tidak ditemukan".

6. Use Case Diagram

6.1 Aktor

Sistem hanya mengenal dua aktor. Tamu adalah pengunjung yang belum login; ia bisa registrasi, login, dan memulihkan password. Pengguna / Admin adalah pengguna yang sudah login (pemilik atau karyawan laundry) di Web Client, dan ia dapat mengelola seluruh data aplikasi. Semua pengguna terdaftar memiliki hak yang sama.

6.2 Diagram



Gambar 6.1 Use Case Diagram LaundryBlackSweet

Pada diagram, use case "Buat pesanan" menyertakan (include) penyimpanan pelanggan baru secara otomatis dan perhitungan total harga otomatis, sedangkan "Cetak struk" memperluas (extend) "Lihat dan cari pesanan". Seluruh fungsi pada diagram disediakan oleh REST API dan diakses lewat Web Client.

6.3 Skenario Use Case

UC-06 Buat Pesanan

Skenario ini dijalankan oleh Pengguna / Admin yang sudah login. Satu syarat lainnya: minimal ada satu layanan yang tersedia.

Alur utama:

1. Pengguna menekan "Pesanan baru".
2. Pengguna mengisi nama, nomor HP, layanan, berat, tanggal, pembayaran, dan catatan.
3. Pengguna menekan simpan, lalu klien mengirim POST /orders.
4. Server memvalidasi input dan mencari harga layanan.
5. Server mencari atau membuat pelanggan (berdasarkan nama + HP).
6. Server menghitung total lalu menyimpan pesanan.
7. Klien memuat ulang daftar, menampilkan toast sukses, lalu membuka dialog Struk untuk pesanan yang baru disimpan.

Alur alternatif. Kalau validasi gagal, server mengembalikan 400 dan pesannya ditampilkan di klien. Bila layanan tidak ditemukan, hasilnya 404; bila token sudah habis, hasilnya 401 dan pengguna diarahkan ke halaman masuk.

Hasil akhir. Pesanan tersimpan dengan status "received", dan pelanggan baru (kalau ada) muncul di menu Pelanggan.

UC-02 Login

Aktornya adalah Tamu, dengan syarat akunnya sudah terdaftar.

Alur utama:

1. Tamu mengisi username dan password.
2. Klien mengirim POST /auth/login.
3. Server memeriksa pembatas percobaan dan memverifikasi password (scrypt).
4. Server membuat token yang berlaku 12 jam.
5. Klien menyimpan token dan menampilkan Beranda.

Alur alternatif. Jika ada field yang kosong, server membalas 400; jika kredensial salah, 401; dan jika percobaan melebihi 10 kali per 15 menit, 429.

Hasil akhir. Pengguna berstatus login, dan token dikirim pada setiap request berikutnya.

UC-08 Ubah Status Pesanan

Aktornya adalah Pengguna / Admin yang sudah login, dan pesanan yang akan diubah sudah ada.

Alur utama:

1. Pengguna memilih status baru pada dropdown di baris pesanan.
2. Klien mengirim PATCH /orders/:id/status.
3. Server memvalidasi status dan memperbarui data.
4. Klien menampilkan lencana status baru dan memperbarui hitungan di Beranda.

Alur alternatif. Status yang tidak sah ditolak dengan 400, pesanan yang tidak ada menghasilkan 404, dan token yang habis menghasilkan 401.

Hasil akhir. Status baru tersimpan, dan pesanan berstatus "completed" muncul di kartu Siap diambil.

UC-11 Hapus Pelanggan

Aktornya adalah Pengguna / Admin yang sudah login, dan pelanggan yang akan dihapus sudah ada.

Alur utama:

1. Pengguna menekan Hapus, lalu dialog konfirmasi muncul.
2. Pengguna menekan tombol Hapus berwarna merah.
3. Klien mengirim DELETE /customers/:id.
4. Server memastikan pelanggan tidak punya pesanan, lalu menghapus datanya.
5. Klien memuat ulang tabel dan menampilkan toast sukses.

Alur alternatif. Jika pelanggan masih punya riwayat pesanan, server menolak dengan 409 dan klien menampilkan toast error.

Hasil akhir. Data pelanggan terhapus, atau tidak berubah sama sekali bila penghapusan ditolak.

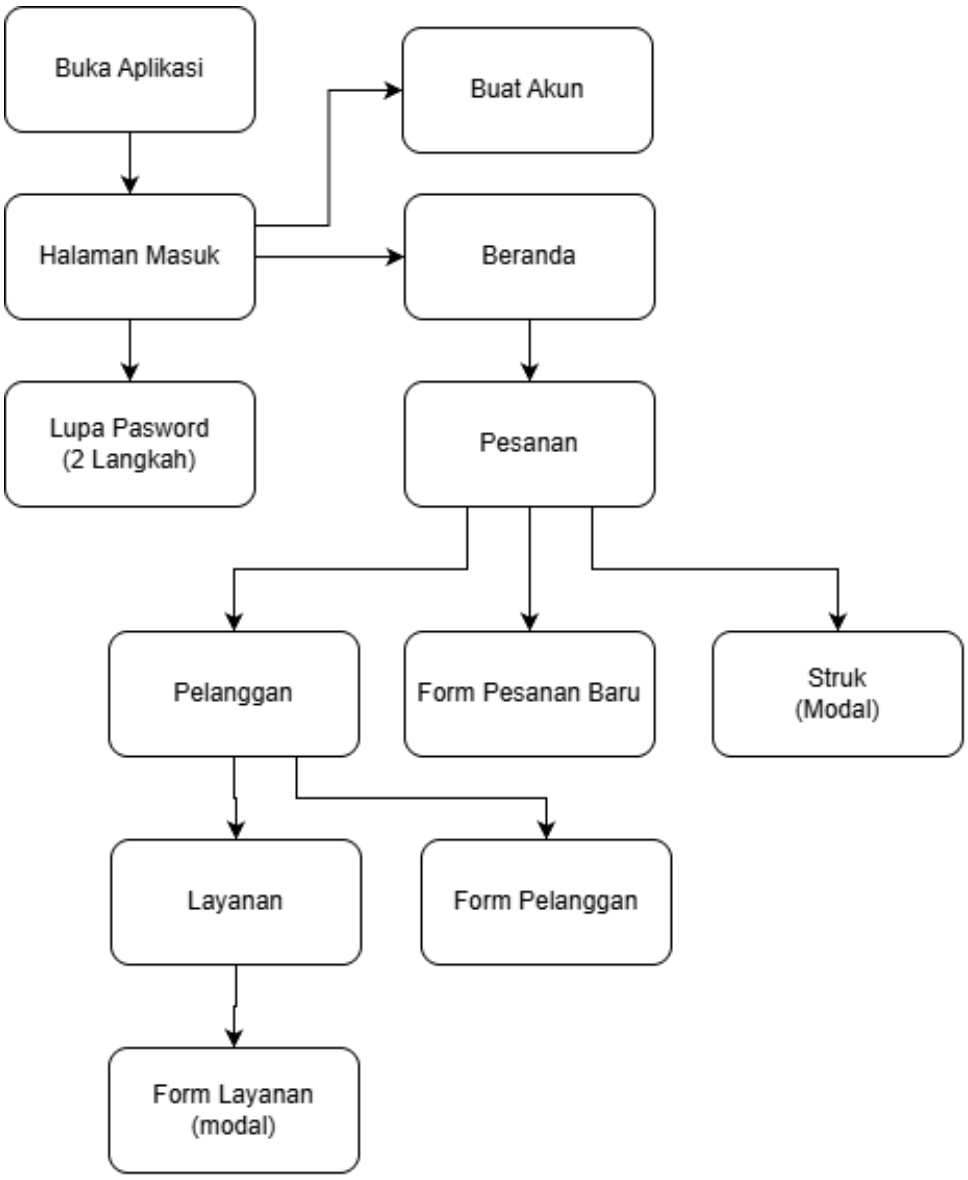
7. UI Design

Bab ini merangkum rancangan antarmuka aplikasi pada tingkat dasar: prinsip desain, identitas visual, komponen, dan tampilan tiap halaman. Seluruh tangkapan layar diambil dari Web Client yang dijalankan dengan kode dan berkas CSS asli aplikasi, memakai data contoh dari seed: lima pelanggan (Adi Wicho Fairuz, Felix Andreas, Tengku Ahmad, Samuel Yogi, dan Agung Sitohang), lima layanan, dan lima pesanan, dengan tanggal acuan 9 November 2026.

7.1 Prinsip Desain

1. **Sederhana dan bersih.** Latar abu muda, kartu putih, dan hanya satu warna aksen, yaitu biru.
2. **Familiar.** Bilah atas, kartu, tombol biru, dan avatar bulat dipilih agar pola interaksinya sudah dikenal pengguna.
3. **Informasi utama di depan.** Angka pendapatan ditampilkan besar, dan status pesanan memakai lencana berwarna.
4. **Umpan balik yang jelas.** Setiap aksi disertai toast, loading, pesan error, atau dialog konfirmasi.
5. **Konsisten.** Warna, jarak, dan bentuk berasal dari satu set variabel CSS.

7.2 Struktur dan Alur Pengguna



Gambar 7.1 Peta navigasi dan alur pengguna

Pengguna yang belum login hanya melihat halaman Masuk. Setelah login, keempat halaman utama bisa dibuka lewat bilah navigasi di atas. Formulir dan konfirmasi muncul sebagai dialog (modal) agar pengguna tidak kehilangan konteks. Untuk mencatat pesanan, pengguna hanya butuh dua klik sebelum mengisi formulir dan satu klik untuk menyimpan.

7.3 Identitas Visual

Identitas visual aplikasi dibangun dari palet yang sengaja dibuat sedikit dan netral, seperti terlihat pada Tabel 7.1.

Tabel 7.1 Palet warna

Warna	HEX	Token CSS	Penggunaan
Abu latar	#F0F2F5	--bg	Latar halaman
Putih kartu	#FFFFFF	--card	Kartu, bilah atas, dialog
Garis	#E4E6EB	--line	Pemisah baris tabel
Teks utama	#050505	--text	Teks isi dan judul
Teks sekunder	#65676B	--muted	Label, keterangan, header tabel
Biru utama	#1877F2	--primary	Tombol utama, tautan, menu aktif
Biru muda	#E7F3FF	--primary-soft	Lencana proses, avatar, kotak total
Merah bahaya	#E41E3F	--danger	Tombol hapus, pesan error
Hijau sukses	#1B8A3A	--ok	Status "Siap diambil", pesan sukses

Untuk huruf, aplikasi memakai huruf sistem ("Segoe UI", Helvetica, Arial, system-ui, sans-serif) dengan ukuran dasar 15 px dan tinggi baris 1,45. Judul halaman berukuran 1,6 rem dengan ketebalan 700, sedangkan judul halaman masuk lebih besar (2,6 rem) dan berwarna biru. Angka statistik juga 1,6 rem, tebal 700, memakai tabular-nums supaya angkanya rapi. Khusus struk, dipakai "Courier New" / monospace agar mirip struk kasir.

Bentuknya dibuat lembut: sudut kartu dan dialog melengkung 8 px, tombol dan input 6 px, sedangkan lencana dan kolom pencarian berbentuk pil. Ikon yang dipakai adalah ikon garis SVG berukuran 22 px dengan gaya seragam untuk Beranda, Pesanan, Pelanggan, dan Layanan.

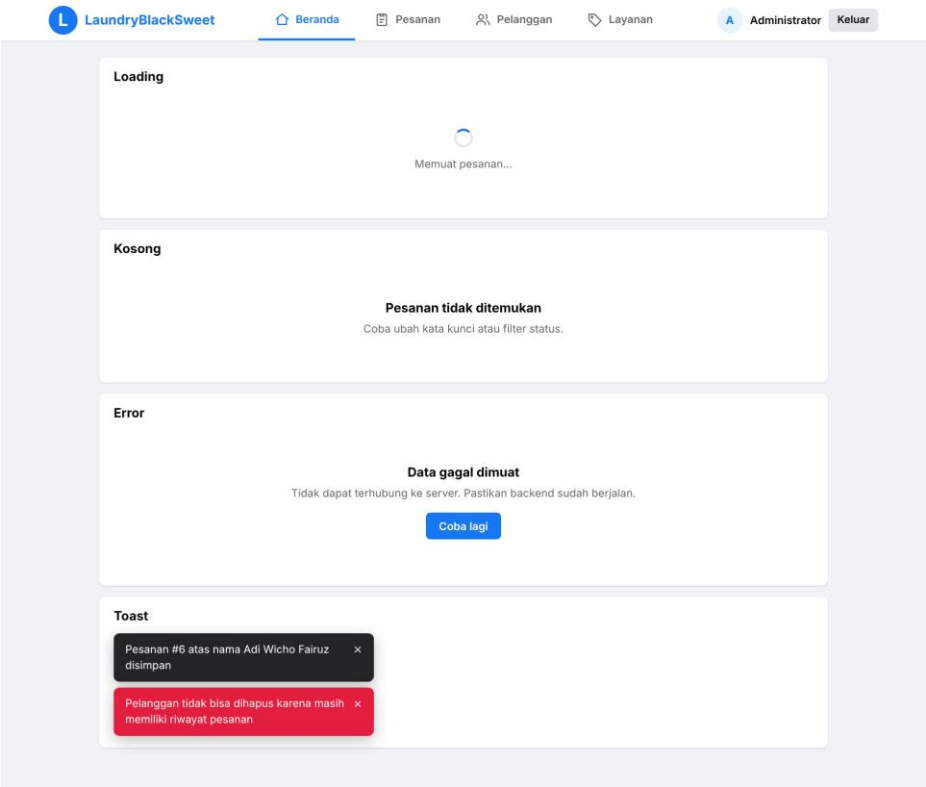
Warna status pesanan dikelompokkan menjadi tiga. Pesanan yang sedang diproses (Diterima, Dicuci, Dikeringkan, Disetrika) berwarna biru, pesanan yang perlu tindakan, yaitu Siap diambil, berwarna hijau, dan pesanan yang sudah selesai (Sudah diambil) berwarna abu.

7.4 Komponen Antarmuka

Antarmuka tersusun dari beberapa komponen dasar. Tombol utama berlatar biru dengan teks putih tebal, menggelap saat di-hover, dan memudar saat nonaktif. Tombol abu dipakai untuk aksi sekunder seperti Batal dan Tutup, sedangkan konfirmasi hapus memakai

tombol merah. Input dan select berlatar abu sangat muda, lalu berubah putih dengan cincin biru saat difokuskan.

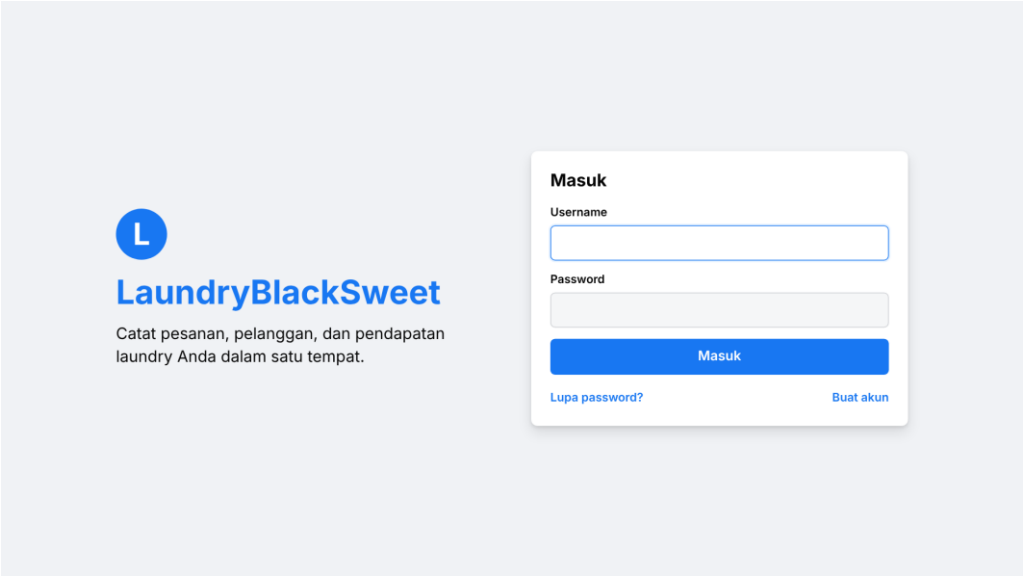
Kartu berwarna putih dengan sudut 8 px. Tabel memiliki header abu berukuran kecil, baris yang menyorot saat di-hover, dan kolom angka rata kanan. Status ditampilkan sebagai pil kecil berwarna sesuai kelompoknya. Dialog (modal) berupa kotak putih di atas lapisan putih semi-transparan yang bisa ditutup dengan tombol, klik di luar dialog, atau tombol Escape. Toast tampil sebagai kotak gelap (sukses) atau merah (error) dan hilang sendiri setelah 3,5 detik. Struk berupa kotak putus-putus berhuruf monospace, mirip struk kasir.



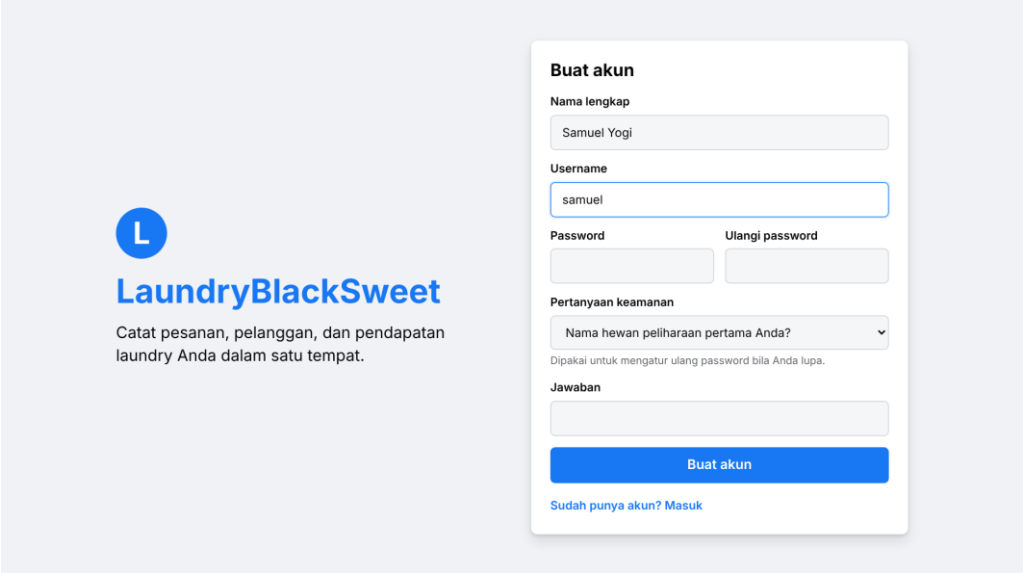
Gambar 7.2 Keadaan antarmuka: loading, kosong, error, dan toast

7.5 Desain Halaman

Halaman Masuk memakai tata letak dua kolom: merek di sebelah kiri dan panel formulir di sebelah kanan. Mode Buat akun dan Lupa password memakai panel yang sama.

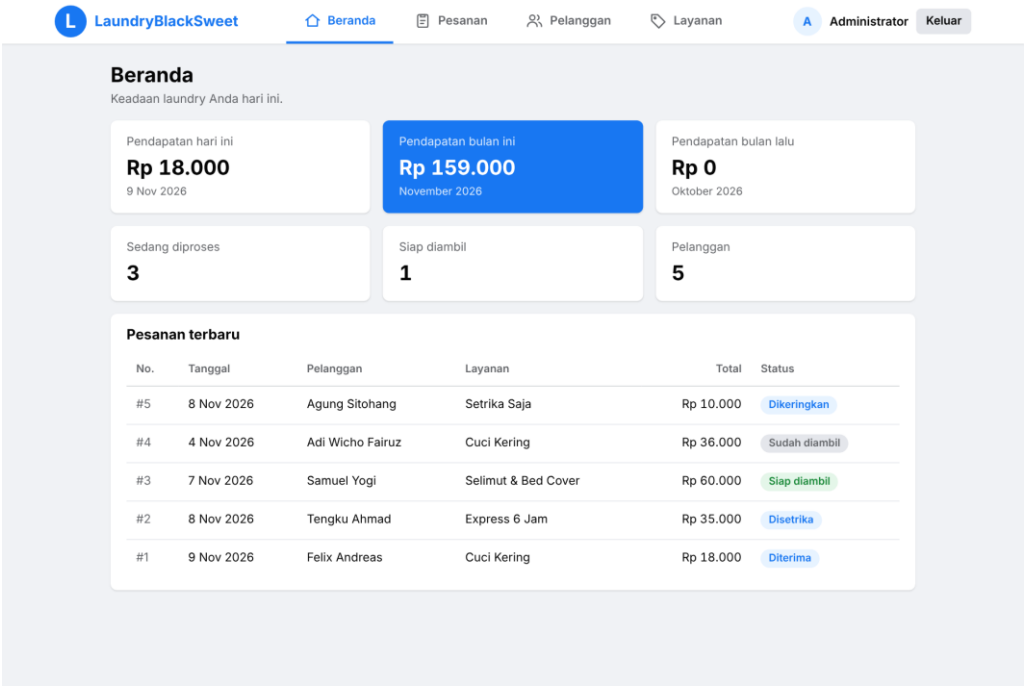


Gambar 7.3 Halaman Masuk



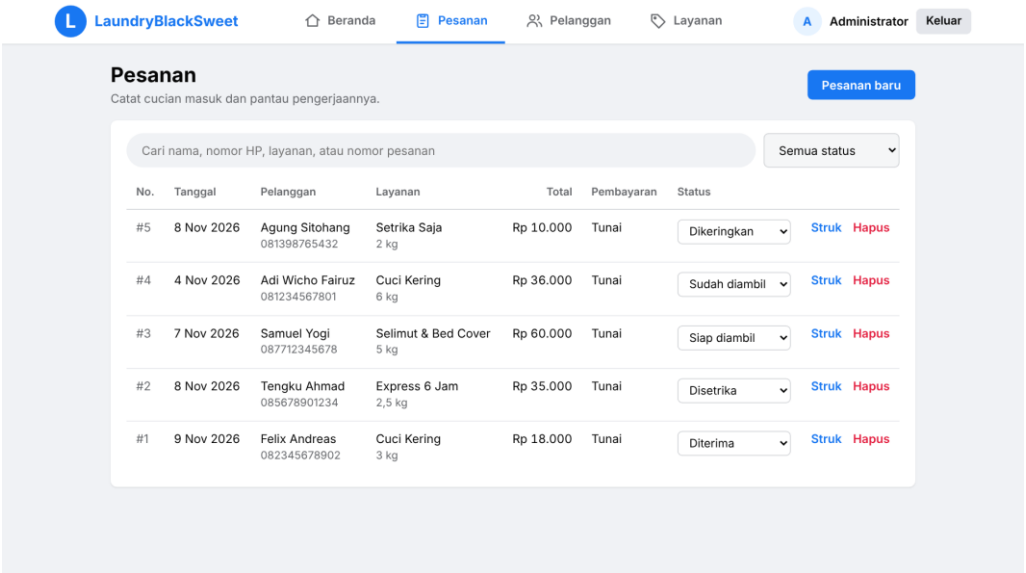
Gambar 7.4 Mode Buat akun

Beranda berisi dua baris kartu statistik tiga kolom (pendapatan dan hitungan) serta tabel delapan pesanan terbaru. Kartu Pendapatan bulan ini diberi latar biru penuh sebagai sorotan utama, sedangkan kartu lainnya berlatar putih.

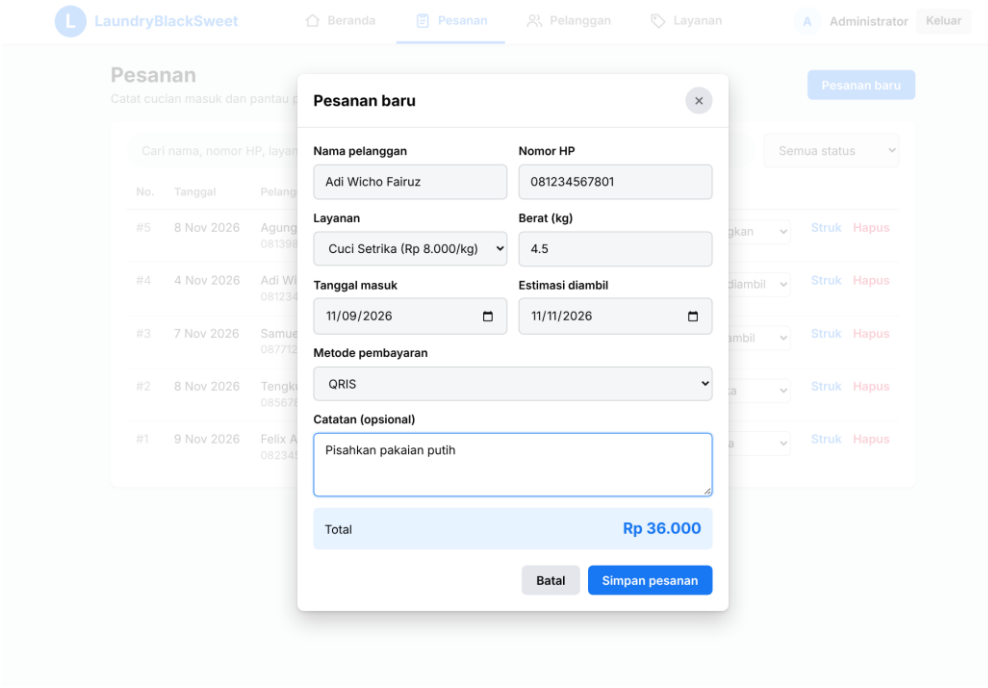


Gambar 7.5 Beranda (dashboard)

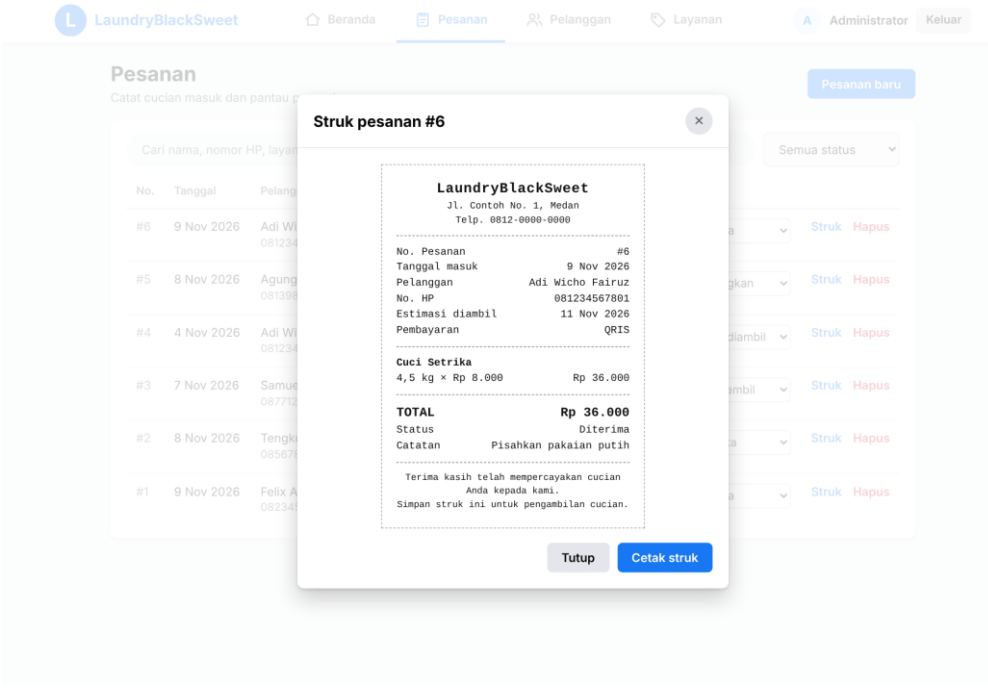
Halaman Pesanan adalah halaman kerja utama, lengkap dengan pencarian, filter status, dropdown ubah status, serta tombol Struk dan Hapus. Pada Gambar 7.7, nama pelanggan yang dipilih dari saran (Adi Wicho Fairuz) langsung mengisi nomor HP-nya, dan setelah pesanan disimpan, dialog Struk (Gambar 7.8) otomatis terbuka agar bisa langsung dicetak.



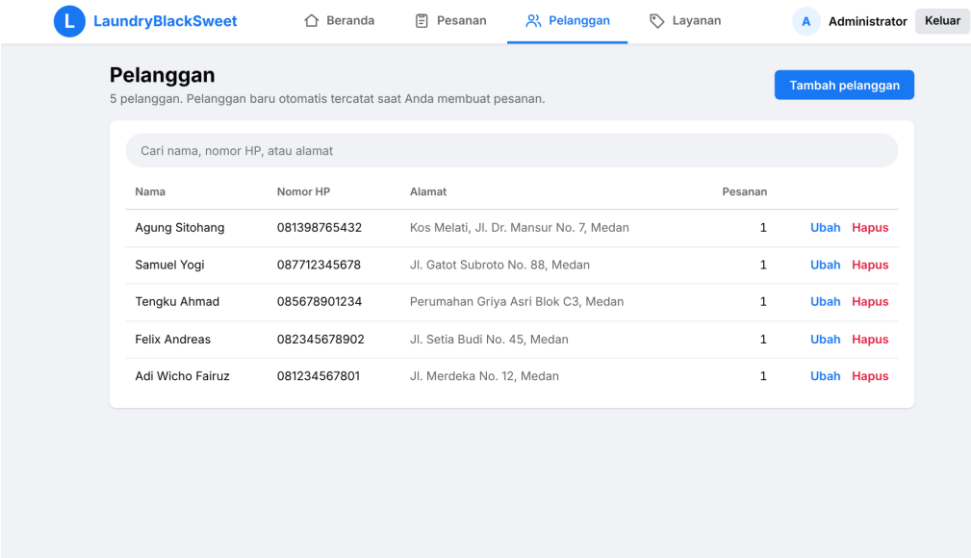
Gambar 7.6 Halaman Pesanan



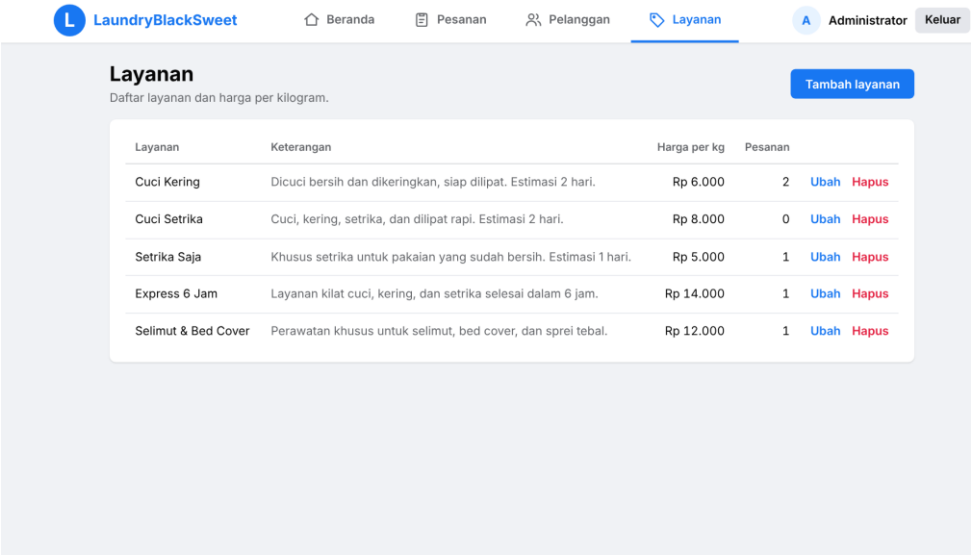
Gambar 7.7 Dialog Pesanan baru dengan kotak total otomatis



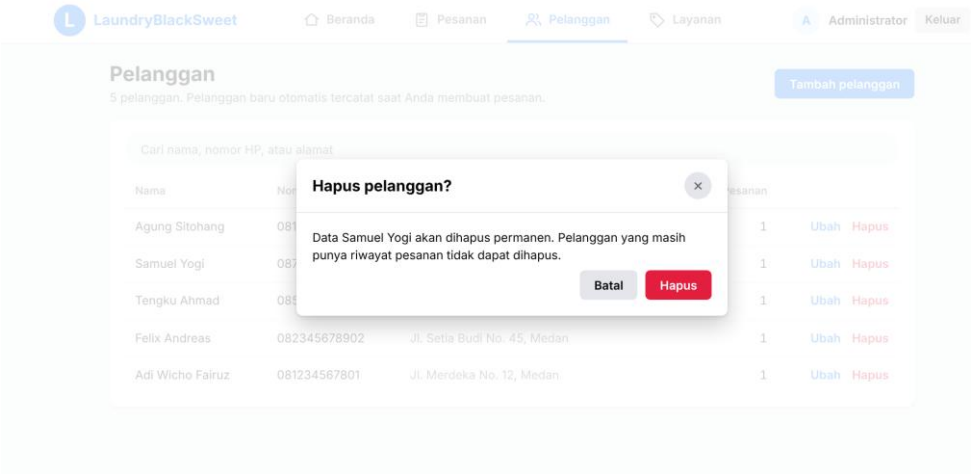
Gambar 7.8 Dialog Struk pesanan



Gambar 7.9 Halaman Pelanggan



Gambar 7.10 Halaman Layanan

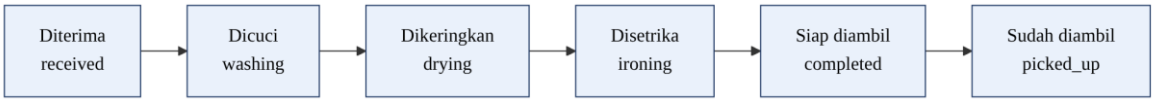


Gambar 7.11 Dialog konfirmasi hapus

8. Flowchart

Bab ini memuat flowchart untuk seluruh proses utama sistem: autentikasi, pembuatan pesanan, pengubahan status, penghapusan data, dan siklus pemrosesan request di server. Pembahasan dimulai dari alur status pesanan.

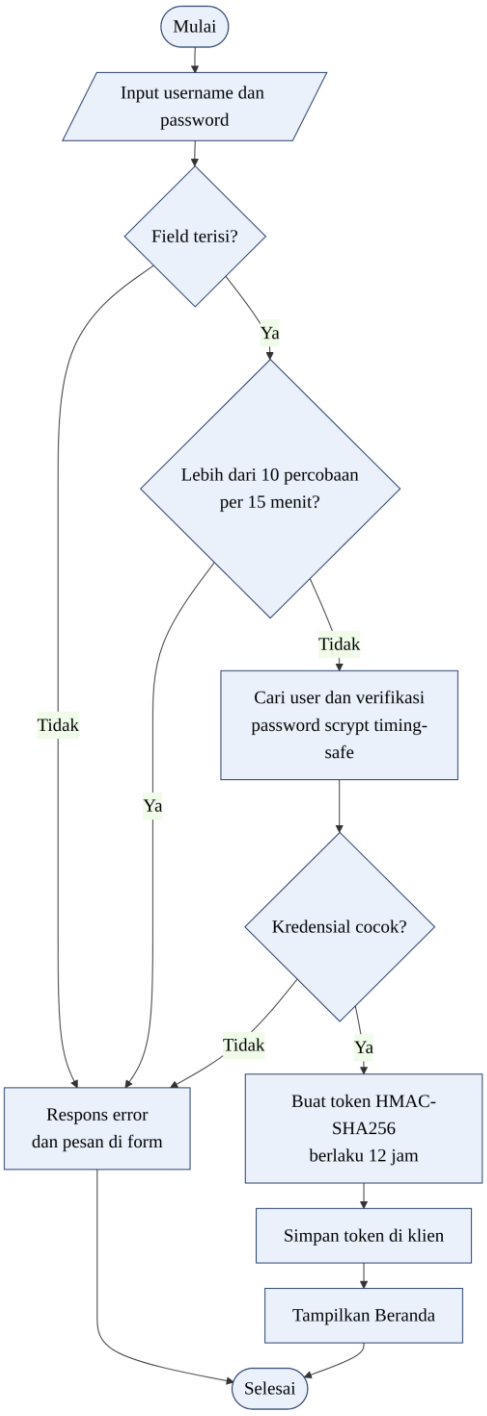
8.1 Alur Status Pesanan



Gambar 8.1 Tahapan status pesanan

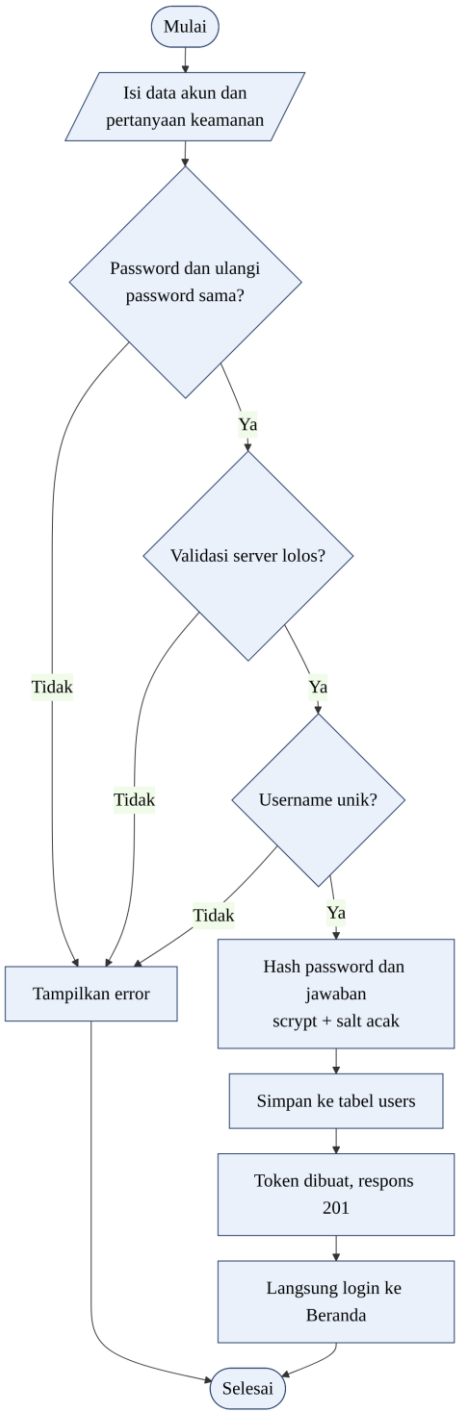
Status disimpan sebagai teks dan divalidasi terhadap enam nilai di atas. Server tidak memaksakan urutan, jadi pengguna bebas memilih status mana pun, misalnya untuk mengoreksi salah klik.

8.2 Flowchart Login



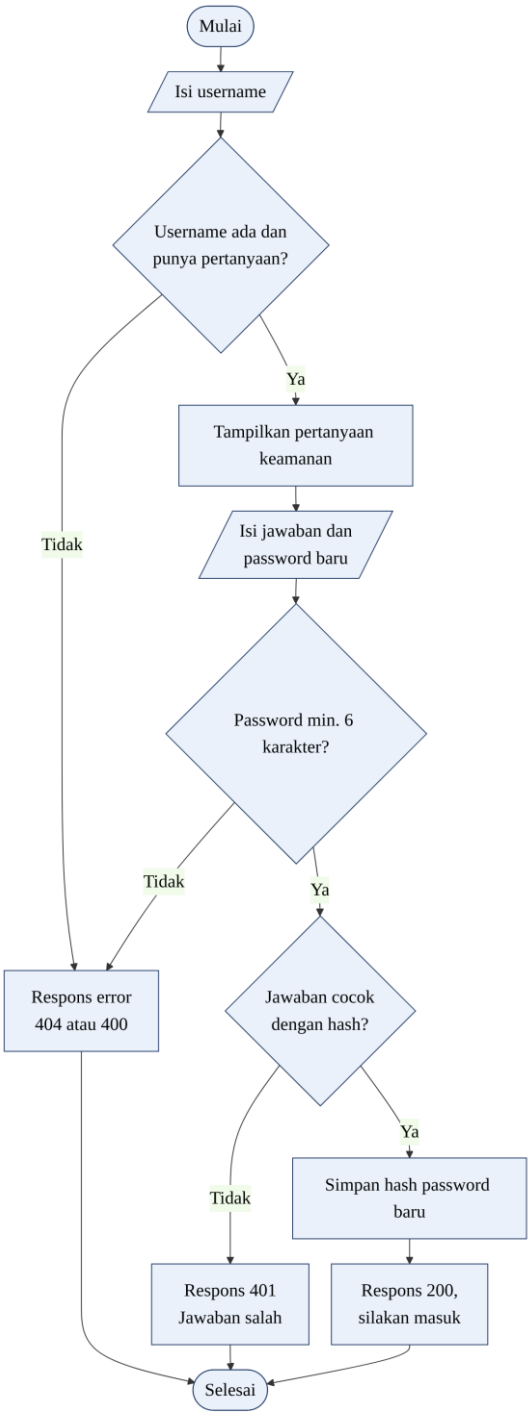
Gambar 8.2 Flowchart proses login

8.3 Flowchart Registrasi Akun



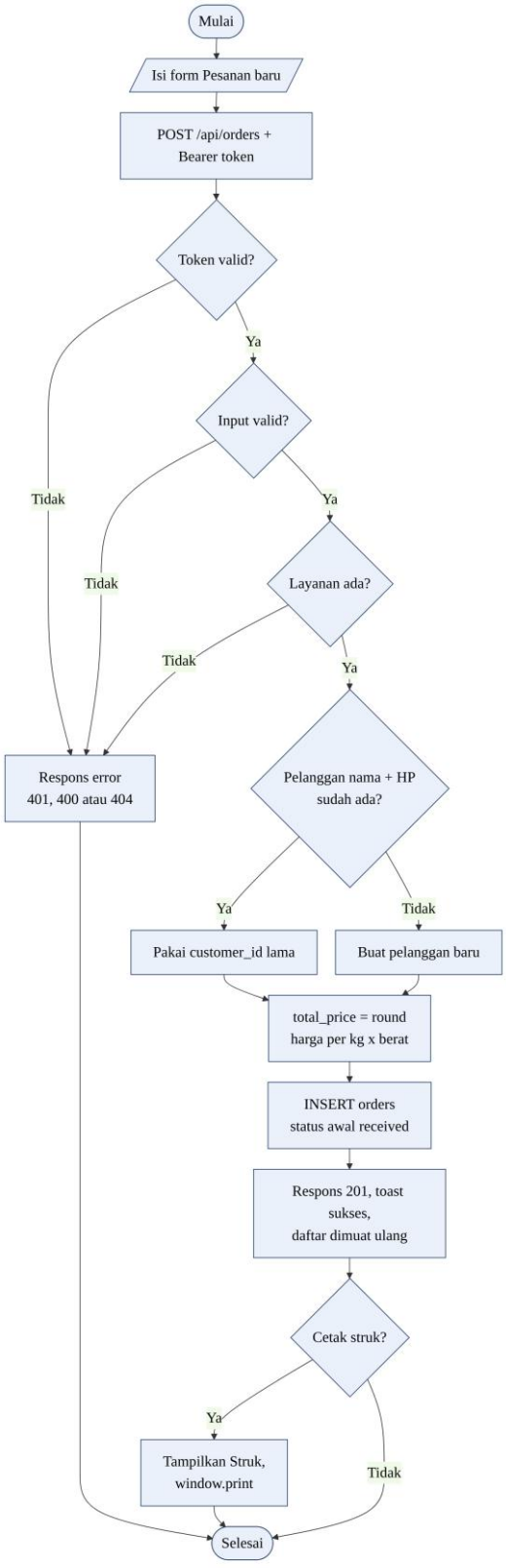
Gambar 8.3 Flowchart proses registrasi akun

8.4 Flowchart Lupa Password



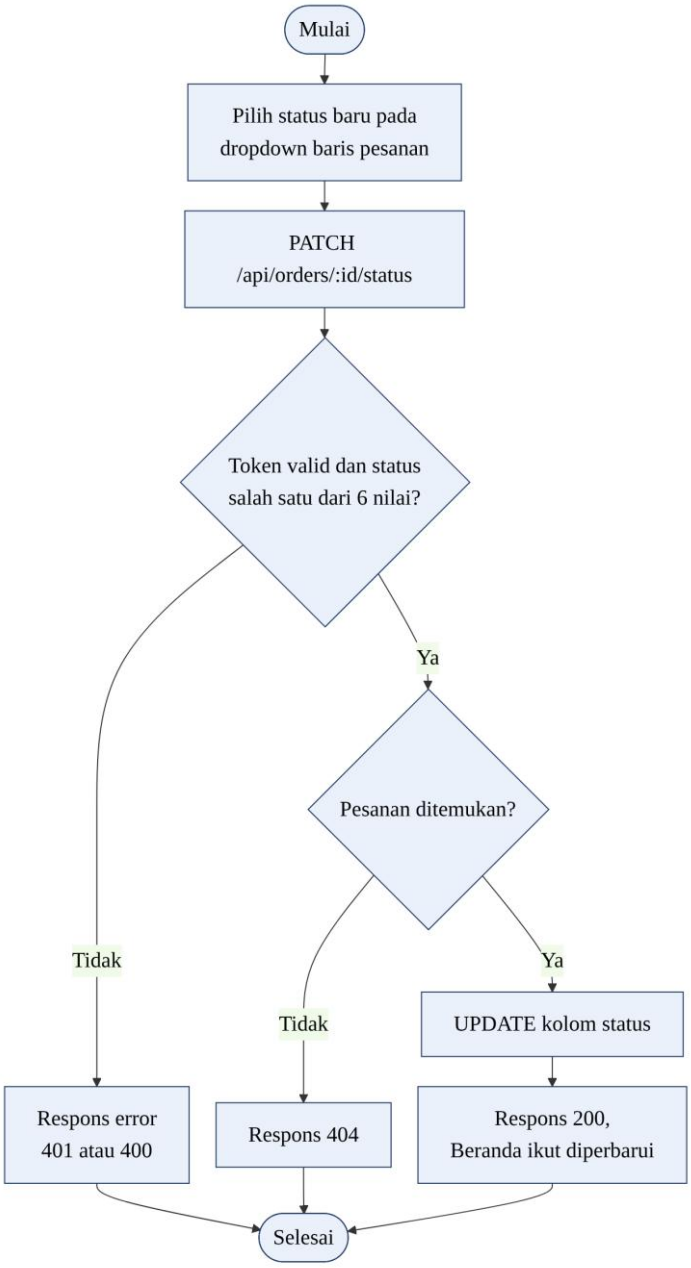
Gambar 8.4 Flowchart pemulihan password

8.5 Flowchart Pembuatan Pesanan



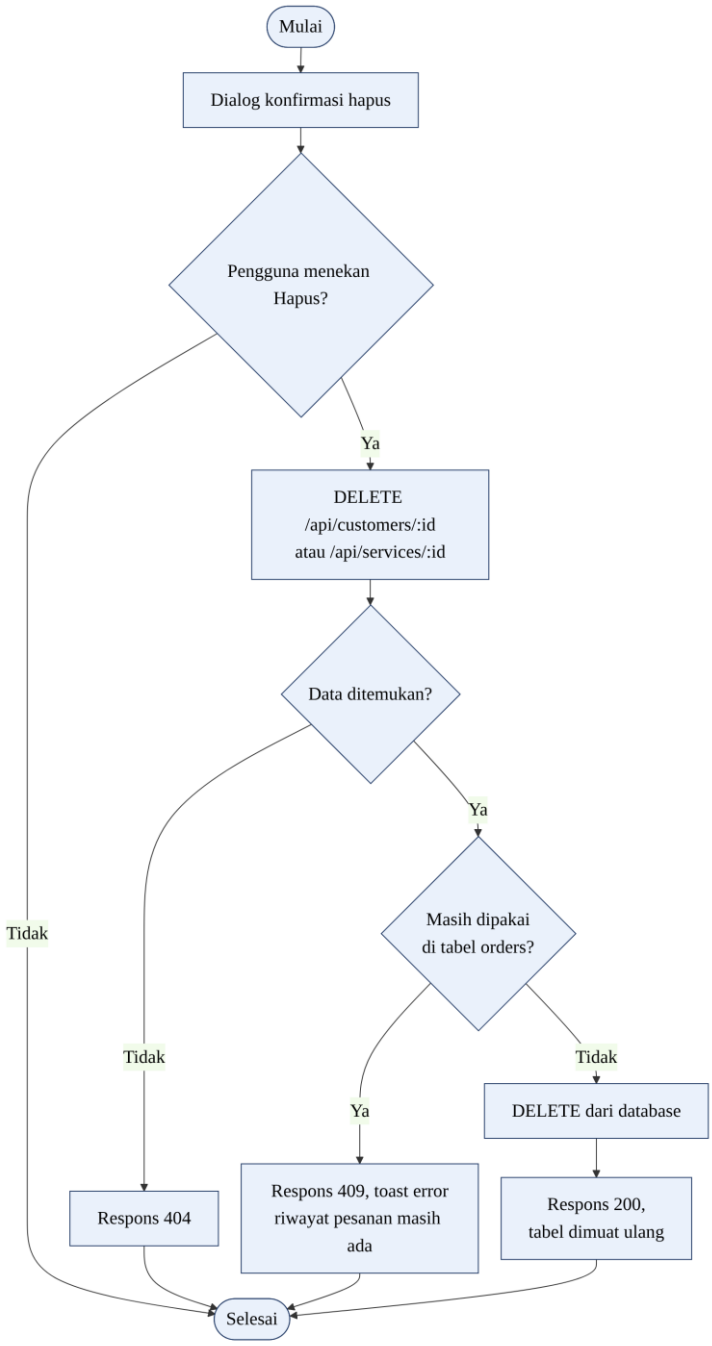
Gambar 8.5 Flowchart pembuatan pesanan

8.6 Flowchart Ubah Status Pesanan



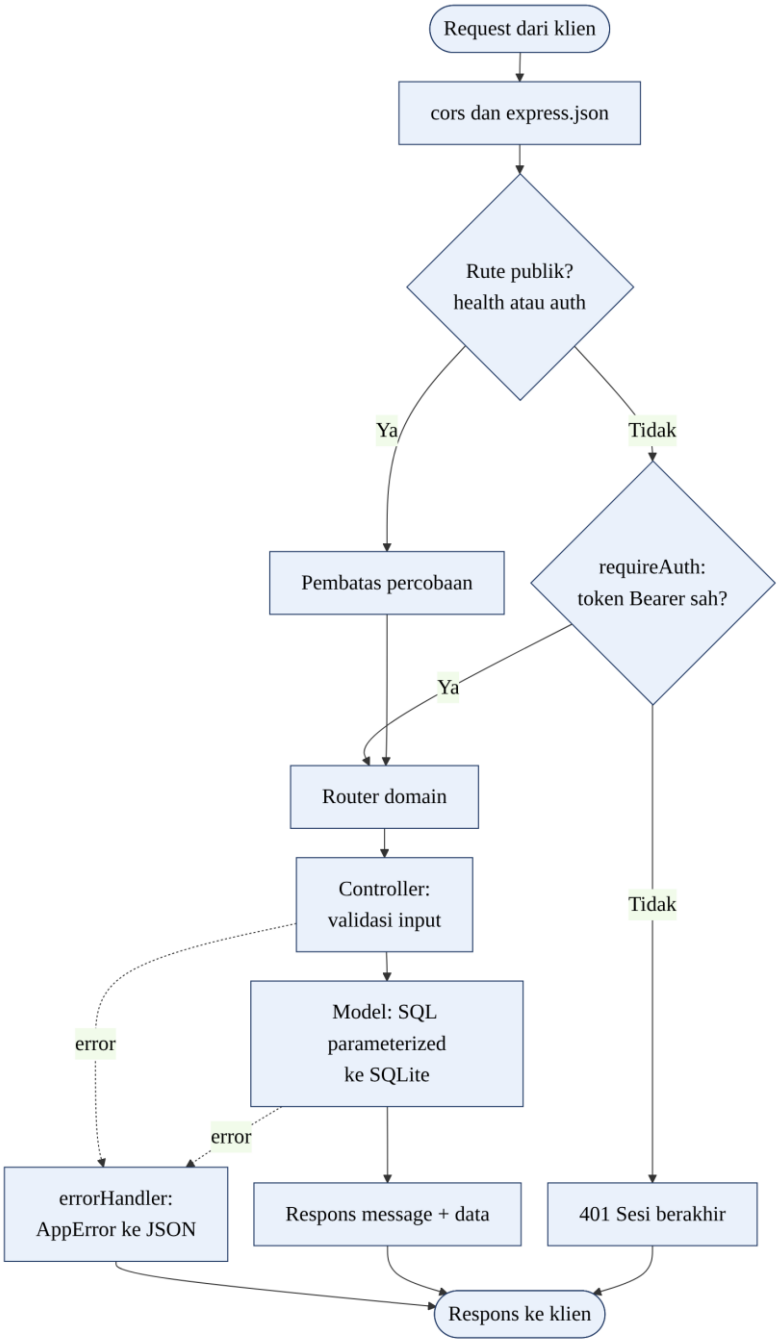
Gambar 8.6 Flowchart ubah status pesanan

8.7 Flowchart Hapus Pelanggan / Layanan



Gambar 8.7 Flowchart penghapusan pelanggan atau layanan

8.8 Flowchart Pemrosesan Request di Server



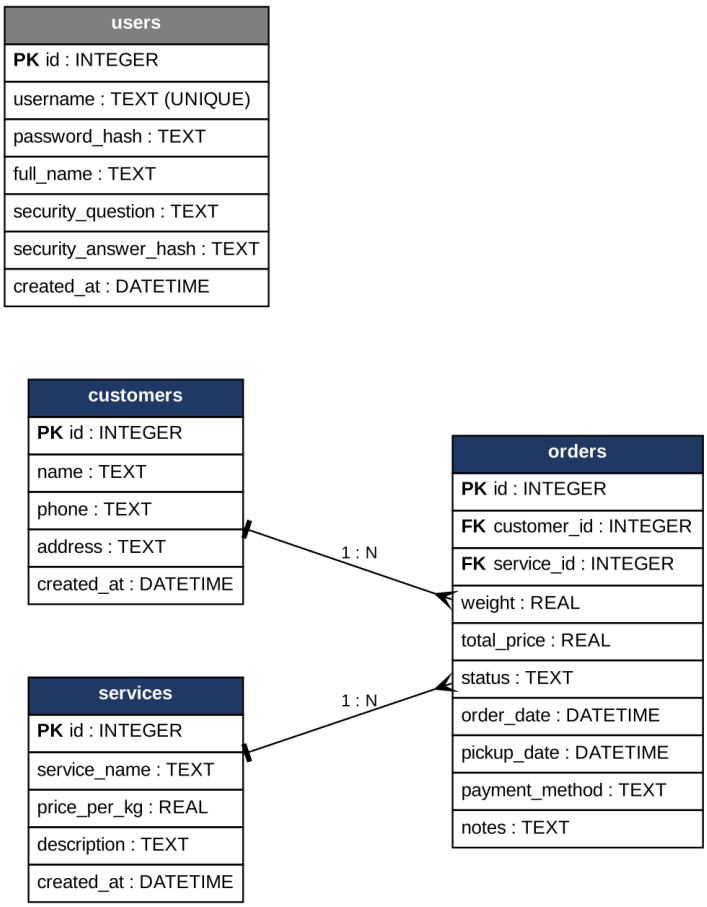
Gambar 8.8 Flowchart siklus pemrosesan request pada server

Setiap request diproses dengan urutan berikut: cors, express.json, lalu rute publik /api/auth (dengan pembatas percobaan) atau requireAuth, kemudian rute domain, controller, model, dan terakhir respons. Error dari lapisan mana pun ditangkap oleh asyncHandler dan diubah oleh errorHandler menjadi respons JSON.

9. Data Model

Basis data memakai SQLite (berkas server/database.sqlite) dengan empat tabel. Tabel-tabel itu dibuat otomatis oleh src/config/database.js saat server pertama kali dijalankan.

9.1 Entity Relationship Diagram



Gambar 9.1 ERD LaundryBlackSweet

Relasi antartabelnya sederhana. Satu pelanggan dapat memiliki banyak pesanan (customers ke orders, 1:N lewat orders.customer_id), dan satu layanan dapat dipakai pada banyak pesanan (services ke orders, 1:N lewat orders.service_id). Tabel users berdiri sendiri karena hanya dipakai untuk autentikasi dan tidak berelasi dengan tabel lain.

9.2 Kamus Data

Berikut kamus data untuk keempat tabel. Tabel 9.1 sampai 9.4 menjelaskan kolom, tipe, constraint, dan keterangan masing-masing.

Tabel customers

Tabel 9.1 Kamus data tabel customers

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PK, AUTOINCREMENT	ID pelanggan
name	TEXT	NOT NULL	Nama pelanggan
phone	TEXT	NOT NULL	Nomor HP
address	TEXT	NOT NULL	Alamat (string kosong bila dibuat otomatis dari pesanan)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Waktu dibuat (UTC)

Tabel services

Tabel 9.2 Kamus data tabel services

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PK, AUTOINCREMENT	ID layanan
service_name	TEXT	NOT NULL	Nama layanan
price_per_kg	REAL	NOT NULL	Harga per kilogram (Rp)
description	TEXT	-	Keterangan layanan
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Waktu dibuat

Tabel orders

Tabel 9.3 Kamus data tabel orders

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PK, AUTOINCREMENT	Nomor pesanan
customer_id	INTEGER	NOT NULL, FK ke customers.id	Pemilik pesanan
service_id	INTEGER	NOT NULL, FK ke services.id	Layanan yang dipilih
weight	REAL	NOT NULL	Berat cucian (kg)
total_price	REAL	NOT NULL	Total harga = round(harga/kg x berat)
status	TEXT	NOT NULL, DEFAULT received	Status pengerjaan
order_date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Tanggal masuk
pickup_date	DATETIME	-	Estimasi tanggal diambil
payment_method	TEXT	NOT NULL, DEFAULT cash	cash / transfer / qris
notes	TEXT	-	Catatan

Tabel users

Tabel 9.4 Kamus data tabel users

Kolom	Tipe	Constraint	Keterangan
id	INTEGER	PK, AUTOINCREMENT	ID pengguna
username	TEXT	NOT NULL, UNIQUE	Nama pengguna
password_hash	TEXT	NOT NULL	Hash scrypt format salt:hash
full_name	TEXT	NOT NULL	Nama lengkap
security_question	TEXT	-	Pertanyaan keamanan
security_answer_hash	TEXT	-	Hash jawaban keamanan
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Waktu dibuat

9.3 Domain Nilai (Enumerasi)

Kolom status pada tabel orders hanya menerima enam nilai: received (Diterima), washing (Dicuci), drying (Dikeringkan), ironing (Disetrika), completed (Siap diambil), dan picked_up (Sudah diambil). Di antarmuka, empat nilai pertama berlabel biru, completed berlabel hijau, dan picked_up berlabel abu.

Kolom payment_method menerima tiga nilai: cash (tunai, menjadi nilai bawaan), transfer (transfer bank), dan qris (pembayaran QRIS).

9.4 Indeks, Integritas, dan Data Awal

- **Indeks.** Ada tiga indeks: idx_orders_customer (orders.customer_id), idx_orders_service (orders.service_id), dan idx_orders_status (orders.status).
- **Foreign key** diaktifkan dengan PRAGMA foreign_keys = ON. Di luar itu, model juga menolak penghapusan pelanggan atau layanan yang masih direferensikan pesanan (HTTP 409).
- **Migrasi ringan.** Fungsi addColumnIfMissing menambahkan kolom payment_method, notes, security_question, dan security_answer_hash pada database lama.
- **Seed.** Bila database kosong, sistem mengisinya dengan 5 pelanggan (Adi Wicho Fairuz, Felix Andreas, Tengku Ahmad, Samuel Yogi, dan Agung Sitohang), 5 layanan (Cuci Kering Rp6.000, Cuci Setrika Rp8.000, Setrika Saja Rp5.000, Express 6 Jam Rp14.000, dan Selimut dan Bed Cover Rp12.000 per kg), serta 5 pesanan contoh (satu pesanan untuk tiap pelanggan) yang bersumber dari server/data.js.
- **Akun awal.** Dibuat otomatis hanya bila tabel users masih kosong (username admin; bisa diubah lewat env ADMIN_USERNAME, ADMIN_PASSWORD, dan ADMIN_SECURITY_ANSWER).

9.5 Analisis Normalisasi

Skema berada pada bentuk normal ketiga (3NF): atribut pelanggan hanya ada di tabel customers, atribut layanan hanya di tabel services, dan tabel orders hanya menyimpan kunci asing beserta data transaksi. Satu-satunya nilai turunan adalah total_price. Kolom ini sengaja disimpan (denormalisasi terkontrol) sebagai catatan harga pada saat transaksi, sehingga riwayat pendapatan tidak ikut berubah ketika harga layanan diubah di kemudian hari.

9.6 Agregasi Data untuk Dashboard

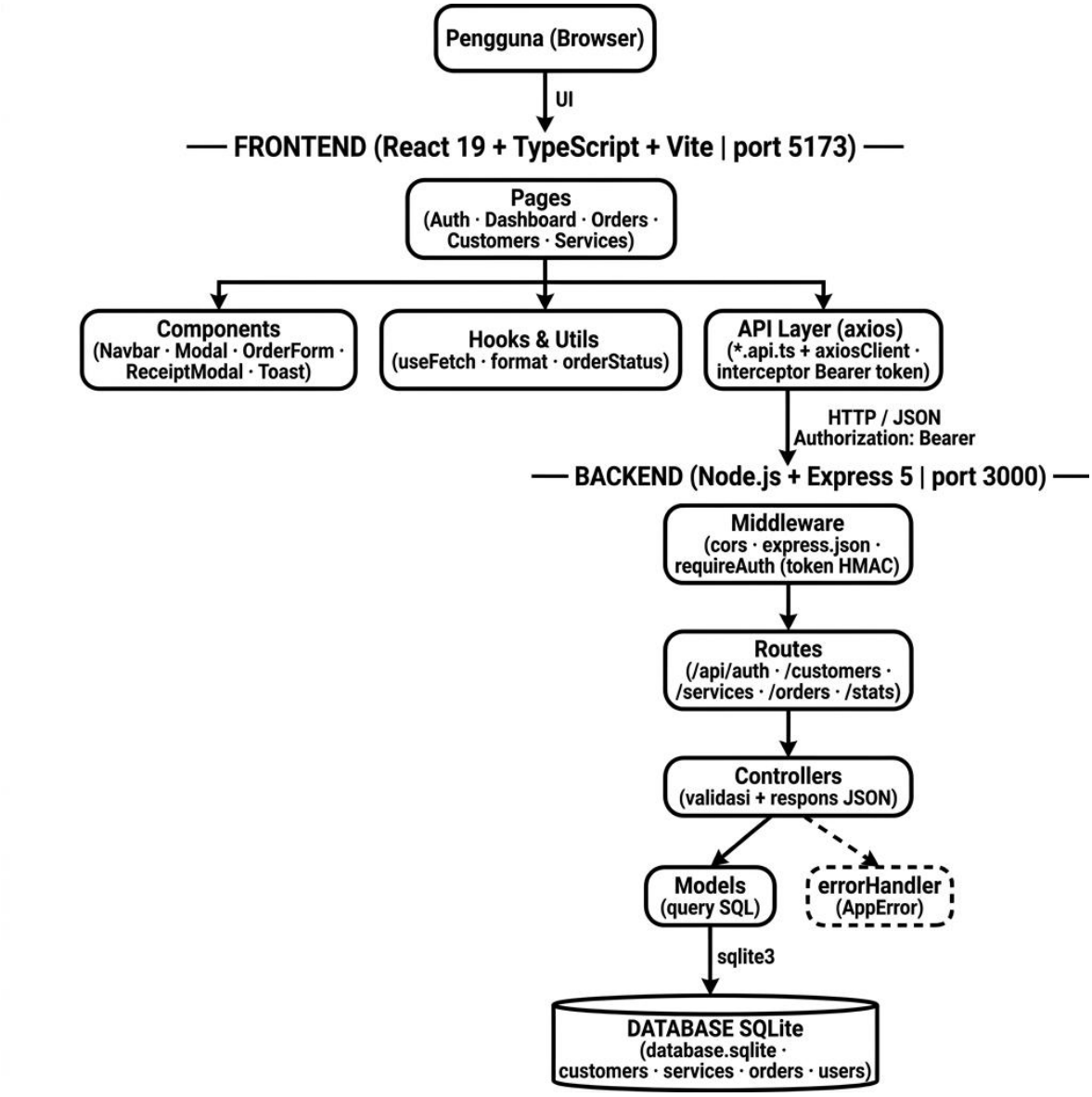
Kartu di Beranda mengambil datanya dari GET /api/stats. active_orders adalah jumlah pesanan yang statusnya bukan completed dan bukan picked_up, sedangkan ready_orders adalah jumlah pesanan berstatus completed. today_revenue, month_revenue, dan last_month_revenue masing-masing menjumlahkan total_price pesanan yang tanggal masuknya jatuh pada hari ini, bulan berjalan, dan bulan sebelumnya. Terakhir, total_customers adalah jumlah baris pada tabel customers.

9.7 Model Data di Sisi Klien (TypeScript)

Antarmuka Order, OrderInput, Customer, Service, Stats, dan AuthUser di src/types mencerminkan bentuk JSON dari API. Enum OrderStatus dan PaymentMethod didefinisikan sebagai tuple konstan, sehingga daftar nilai dan tipenya berasal dari satu sumber.

10. Architecture Diagram

10.1 Arsitektur Sistem



Gambar 10.1 Arsitektur LaundryBlackSweet (client-server tiga lapis, klien web)

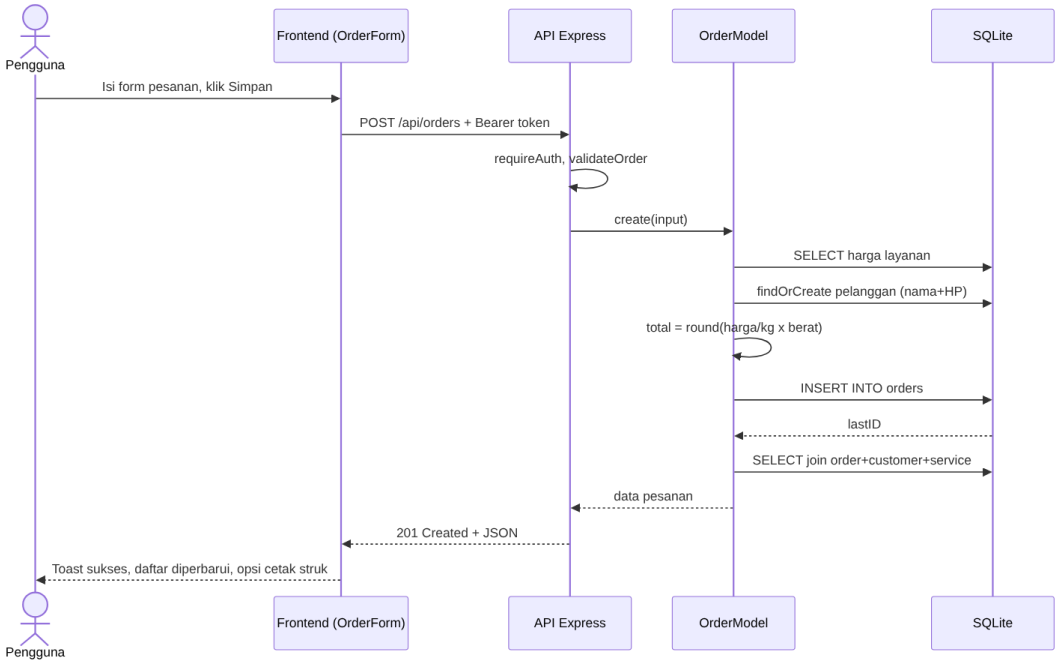
Sistem memakai arsitektur client-server. Single Page Application React berjalan di browser (port 5173 saat pengembangan) dan berkomunikasi dengan REST API Express (port 3000) lewat HTTP/JSON. Backend-nya berlapis: middleware, routes, controllers, models, lalu basis data SQLite.

10.2 Lapisan Backend

Backend disusun berlapis, dan setiap lapisan punya tugas yang jelas:

1. **app.js / server.js** membuat aplikasi Express dan memasang cors, express.json, rute, requireAuth, notFound, dan errorHandler. server.js menjalankan listener pada PORT (bawaan 3000).
2. **middleware/auth.js** berisi requireAuth, yang membaca header Bearer, memverifikasi token, dan mengisi req.user, atau mengembalikan 401.
3. **middleware/errorHandler.js** berisi notFound (404) dan errorHandler, yang menerjemahkan AppError, JSON rusak, dan error tak terduga menjadi respons JSON.
4. **routes/** memetakan URL dan method ke controller (auth, customers, services, orders, stats).
5. **controllers/** mengambil input, memanggil validator dan model, lalu mengembalikan respons {message, data}.
6. **model/** menampung semua query SQL (parameterized) dan aturan data, misalnya findOrCreate pelanggan, penghitungan total, dan larangan menghapus data yang masih dipakai.
7. **utils/** berisi asyncHandler, AppError, validators, auth (scrypt dan token HMAC), serta dbAsync (pembungkus Promise untuk sqlite3).
8. **config/database.js** mengurus koneksi SQLite, pembuatan tabel, indeks, migrasi, dan seed.

10.3 Alur Pembuatan Pesanan



Gambar 10.2 Sequence diagram pembuatan pesanan

10.4 Arsitektur Keamanan

1. Password dan jawaban keamanan di-hash dengan scrypt (64 byte) memakai salt acak 16 byte.
2. Token berisi payload JSON (id, username, full_name, exp) yang di-encode base64url lalu ditandatangani HMAC-SHA256 dengan AUTH_SECRET, dan dibandingkan dengan crypto.timingSafeEqual.
3. Pembatas percobaan disimpan di memori per IP dan jenis aksi (login, register, forgot): 10 kali per 15 menit.
4. Query SQL memakai placeholder (?) sehingga aman dari SQL injection.
5. Di sisi klien, token disimpan di localStorage browser.

10.5 Arsitektur Penerapan

Pada lingkungan pengembangan, backend Express berjalan di localhost:3000, basis data SQLite tersimpan di server/database.sqlite, dan Web Client berjalan di localhost:5173 lewat Vite.

Untuk produksi, ada beberapa hal yang perlu disiapkan. Backend dijalankan dengan npm start di balik reverse proxy HTTPS, dan berkas SQLite sebaiknya dicadangkan secara berkala. Web Client dibuild menjadi berkas statis (npm run build) yang disajikan oleh web server.

11. Integration Doc

Bab ini berisi panduan untuk menjalankan, menghubungkan, dan menguji Web Client dengan backend, serta untuk mengintegrasikan klien lain ke API ini.

11.1 Prasyarat

1. Node.js (disarankan versi LTS terbaru) dan npm.
2. Port 3000 (backend) dan 5173 (frontend web) dalam keadaan bebas.

11.2 Langkah Menjalankan

1. **Backend.** Buka terminal, lalu jalankan cd server, npm install, dan npm run dev (atau npm start). Database database.sqlite dibuat dan diisi data awal secara otomatis.
2. **Web.** Buka terminal baru, lalu jalankan cd frontend, npm install, dan npm run dev.
3. **Akses.** Buka http://localhost:5173 dan masuk dengan akun awal admin / admin123.
4. **Verifikasi.** Buka http://localhost:3000/api/health; respons yang diharapkan adalah { "status": "OK", ... }.

11.3 Konfigurasi

Perilaku backend dan klien bisa diatur lewat beberapa variabel konfigurasi, seperti pada Tabel 11.1.

Tabel 11.1 Variabel konfigurasi

Variabel	Lokasi	Fungsi
VITE_API_URL	frontend/.env	Alamat API untuk klien web (bawaan http://localhost:3000/api)
PORT	environment server	Port backend (bawaan 3000)
AUTH_SECRET	environment server	Rahasia penanda tangan token; WAJIB diganti saat produksi
ADMIN_USERNAME	environment server	Username admin awal (bawaan admin)
ADMIN_PASSWORD	environment server	Password admin awal (bawaan admin123)
ADMIN_SECURITY_ANSWER	environment server	Jawaban keamanan admin awal (bawaan laundryblacksweet)

Sebagai contoh, backend bisa dijalankan dengan konfigurasi sendiri seperti ini:

```
ADMIN_USERNAME=pemilik ADMIN_PASSWORD=passwordkuat \
ADMIN_SECURITY_ANSWER=jawabanrahasia AUTH_SECRET=teks-acak-panjang npm start
```

11.4 Alur Integrasi Autentikasi

1. Klien memanggil POST /api/auth/login (atau /register) dan menerima data.token.
2. Token disimpan di localStorage browser.
3. Setiap request berikutnya menyertakan header Authorization: Bearer <token>.
4. Bila server membalas 401, klien menghapus token dan meminta pengguna login ulang.

11.5 Contoh Pemanggilan dengan cURL

```
# 1. Login dan ambil token
curl -X POST http://localhost:3000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123"}'

# 2. Daftar layanan
curl http://localhost:3000/api/services -H "Authorization: Bearer $TOKEN"

# 3. Buat pesanan
curl -X POST http://localhost:3000/api/orders \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"customer_name":"Adi Wicho
Fairuz","customer_phone":"081234567801","service_id":2,"weight":3,"payment_method":"cash"}'

# 4. Ubah status
curl -X PATCH http://localhost:3000/api/orders/6/status \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"status":"completed"}'
```

11.6 Contoh Integrasi Klien JavaScript

```
const API = "http://localhost:3000/api";

async function api(path, options = {}, token) {
  const res = await fetch(API + path, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
    },
  });
  const json = await res.json();
  if (!res.ok) throw new Error(json.message);
  return json.data;
}

// penggunaan
const { token } = await api("/auth/login", {
  method: "POST", body: JSON.stringify({ username: "admin", password: "admin123" }),
});
const orders = await api("/orders", {}, token);
```

11.7 Pengujian Integrasi

Tabel 11.2 memuat sepuluh skenario untuk memastikan integrasi berjalan benar.

Tabel 11.2 Skenario uji integrasi

No.	Skenario	Hasil yang diharapkan
1	GET /api/health tanpa token	200 { "status": "OK" }
2	GET /api/orders tanpa token	401 Sesi berakhir
3	Login admin / admin123	200 dan token
4	Login dengan password salah 11 kali	401 lalu 429 pada percobaan berikutnya
5	POST /api/orders dengan berat 0	400 Berat cucian harus antara 0 dan 1000 kg
6	POST /api/orders sah (layanan 2, berat 4,5)	201 dan total_price sesuai harga x berat
7	PATCH status dengan nilai "foo"	400 Status pesanan tidak valid
8	DELETE layanan yang sudah dipakai pesanan	409
9	Token kedaluwarsa pada klien	Klien kembali ke halaman Masuk
10	GET /api/orders dibandingkan dengan isi tabel orders	Data identik

11.8 Penanganan Masalah Umum

Beberapa masalah yang paling sering muncul beserta penyelesaiannya:

1. **Network Error di klien.** Biasanya backend belum berjalan atau base URL salah. Jalankan backend, lalu periksa VITE_API_URL.
2. **Selalu 401 setelah login.** Penyebabnya bisa AUTH_SECRET yang berubah atau token yang sudah kedaluwarsa. Login ulang, dan pastikan AUTH_SECRET tidak berganti.
3. **429 Terlalu banyak percobaan.** Pembatas percobaan per IP sedang aktif. Tunggu 15 menit, atau restart server (hanya di lingkungan pengembangan).

12. Feature Depth

Bab ini menguraikan kedalaman fitur LaundryBlackSweet pada tingkat lanjut: bukan sekadar operasi CRUD, tetapi juga aturan bisnis, keamanan, ketahanan, dan pengalaman pengguna.

12.1 Fitur Lanjutan Sisi Server

Perhitungan harga sepenuhnya dilakukan di server: total_price = round(harga per kg × berat) dihitung saat pesanan dibuat, sedangkan klien hanya menampilkan estimasi. Hasilnya konsisten dan tidak bisa dimanipulasi dari sisi klien. Pelanggan dicari lewat nama dan nomor HP, lalu dibuat otomatis bila belum ada (findOrCreate), sehingga data tidak ganda dan input pesanan lebih cepat. Integritas data dijaga dengan menolak penghapusan pelanggan atau layanan yang masih dipakai pesanan (409), ditambah foreign key yang aktif, sehingga riwayat transaksi tidak rusak.

Dari sisi keamanan, password dan jawaban keamanan tidak disimpan sebagai teks asli, melainkan sebagai hash scrypt dengan salt, jadi tetap aman bila basis data bocor. Token ditandatangani dengan HMAC-SHA256, kedaluwarsa dalam 12 jam, dan diverifikasi secara timing-safe, sehingga autentikasi bersifat stateless tanpa tabel sesi. Rate limiting (10 percobaan per 15 menit per IP untuk login, register, dan forgot) mencegah tebak password, dan pemulihan password lewat pertanyaan keamanan dua langkah tanpa email membuat akun bisa dipulihkan secara mandiri.

Validasi dilakukan berlapis: nama, nomor HP (regex 8-16 digit), berat (0-1000 kg), enum status dan pembayaran, serta format tanggal, sehingga pesan error spesifik dan data tetap bersih. Penanganan error dipusatkan lewat asyncHandler, AppError, errorHandler, dan notFound (termasuk untuk JSON rusak), sehingga semua respons error seragam. Migrasi ringan (addColumnIfMissing), data awal, dan akun admin dari env membuat instalasi tidak memerlukan langkah manual. Endpoint statistik menggabungkan pendapatan hari ini, bulan ini, bulan lalu, serta pesanan aktif dan siap diambil dalam satu request, dan query terparameter dengan tiga indeks pada tabel orders menjaga sistem tetap aman dari SQL injection dan cepat saat memfilter.

12.2 Fitur Lanjutan Sisi Klien

Di sisi klien, hook useFetch (dengan reload dan retry) menyeragamkan pola loading, error, dan data, serta memungkinkan muat ulang tanpa berkedip. Interceptor menempelkan token secara otomatis, dan respons 401 memicu event logout sehingga pengguna kembali ke halaman masuk. Pada form pesanan, kotak total menghitung harga langsung saat layanan dan berat diubah, sementara nama pelanggan yang sudah ada disarankan untuk mengisi nomor HP secara otomatis.

Daftar pesanan bisa dicari berdasarkan nama, HP, layanan, atau nomor, dan disaring menurut enam status. Status juga bisa diubah langsung lewat dropdown di baris tabel tanpa membuka halaman lain. Struk dibuat bergaya kasir selebar 80 mm dengan aturan @media print. Selain itu, navigasi memakai hash sehingga halaman aktif tetap saat di-refresh tanpa router eksternal; aksesibilitas dijaga lewat label terhubung, role dialog, aria-live, fokus keyboard, dan dukungan prefers-reduced-motion; React Compiler mengoptimalkan render secara otomatis pada build Vite; dan seluruh logika bisnis berada di server sehingga klien cukup menampilkan data dan mengirim input.

12.3 Aturan Bisnis Utama

Ada tujuh aturan bisnis yang menjadi pegangan sistem:

1. **BR-01.** Total harga dihitung sebagai round(harga per kg × berat), dan berat harus lebih dari 0 serta maksimal 1000 kg.
2. **BR-02.** Pelanggan dianggap unik berdasarkan kombinasi nama dan nomor HP.
3. **BR-03.** Status pesanan hanya boleh salah satu dari enam nilai yang sah; urutannya tidak dipaksakan.
4. **BR-04.** Pelanggan atau layanan yang sudah punya pesanan tidak boleh dihapus.
5. **BR-05.** Pesanan baru otomatis berstatus received dengan metode pembayaran cash bila tidak diisi.
6. **BR-06.** Pendapatan dihitung dari total_price yang tersimpan, bukan dari harga layanan terkini.
7. **BR-07.** Semua endpoint selain auth dan health memerlukan token yang belum kedaluwarsa.

Bab 14 Rencana Pengembangan Mobile dan Desktop

14.1 Gambaran

LaundryBlackSweet akan dikembangkan ke platform Mobile dan Desktop tanpa mengubah logika bisnis, karena seluruh aturan (hitung harga, validasi, larangan hapus data terpakai) sudah berada di REST API Express. Aplikasi Web Client saat ini sudah berjalan penuh; Mobile dan Desktop berstatus rencana lanjutan dan belum diimplementasikan.
Tiga prinsip yang dipegang:

1. Satu API untuk semua klien. Mobile dan Desktop memakai 21 endpoint yang sama dengan format respons `{ message, data }` dan autentikasi Bearer.
2. Logika bisnis tetap di server. Klien baru hanya menampilkan data dan mengirim input, sehingga total harga tetap konsisten di semua platform.
3. Berbagi kode bila memungkinkan. Tipe data TypeScript (`src/types`) dan lapisan API (`src/api`) pada Web Client dipakai ulang agar kontrak data tidak bercabang.

14.2 Aplikasi Mobile

Aplikasi Mobile ditujukan bagi karyawan dan pemilik yang bekerja di luar meja kasir, dibangun dengan React Native (Expo) agar tipe TypeScript dan lapisan API dari Web Client bisa dipakai ulang di Android dan iOS.

Aspek	Rencana
Pengguna	Pemilik dan karyawan laundry
Teknologi	React Native + Expo, TypeScript, axios
Penyimpanan token	expo-secure-store (menggantikan localStorage)
Fitur awal	Login, beranda pendapatan, daftar dan ubah status pesanan, buat pesanan, kelola pelanggan dan layanan
Fitur khusus perangkat	Pindai QR/barcode nota, kirim struk lewat tombol bagikan, cetak ke printer Bluetooth
Pemakaian API	Endpoint yang sama; VITE_API_URL diganti konfigurasi alamat server
Mode luring	Antrean pesanan lokal yang dikirim saat koneksi kembali (tahap lanjutan)

14.3 Aplikasi Desktop

Aplikasi Desktop ditujukan bagi kasir di komputer toko

Aspek	Rencana
Pengguna	Kasir di komputer toko
Teknologi	Electron + Vite/React (kode Web Client yang sama)
Platform	Windows (utama), macOS dan Linux (opsional)
Fitur awal	Seluruh fitur Web Client dalam jendela aplikasi
Fitur khusus perangkat	Cetak struk langsung ke printer
Penyimpanan token	Penyimpanan aman sistem operasi (safeStorage Electron)
Pemakaian API	Endpoint yang sama; alamat server diatur lewat berkas konfigurasi
Pembaruan	Pembaruan otomatis lewat electron-updater

14. Penutup

14.1 Kesimpulan

LaundryBlackSweet menjawab masalah pencatatan laundry manual lewat satu REST API yang aman dan konsisten, ditambah klien web yang sudah berfungsi penuh. Perhitungan harga di server, findOrCreate pelanggan, perlindungan data yang masih dipakai, dan autentikasi berlapis membuat data tetap akurat dan terlindungi. Dokumentasi dalam laporan ini mencakup seluruh komponen penilaian kelompok lima orang: analisis kebutuhan, use case, flowchart dan model data yang lengkap, arsitektur, dokumentasi API, dan panduan integrasi.